

**Final Year Project for the Master's Degree in Computer
Science**

Specialty: Big Data Analytics

Topic

**Design and Implementation of a Prototype of a
Distributed Storage System in a Mobile
Environment**

Supervised by:
Mr. DJOUAMA Amir

Presented by:
NAOUI Tarek Anis
NEMIRI Sara Yasmine

Defense date: 13/06/2026

Pair No. 2

Acknowledgments

We would like to express our sincere gratitude to everyone who contributed to the completion of this thesis.

First and foremost, we would like to extend our deepest appreciation to our supervisor, Amir DJOUAMA, for his valuable guidance, insightful advice, and continuous support throughout this work. His expertise, patience, and encouragement played a fundamental role in the successful realization of this study.

We also wish to thank the entire Computer Science Department of ESST for the quality of education provided and for their constant support during our academic journey.

Finally, we would like to express our heartfelt gratitude to our families and friends for their unconditional support, understanding, and encouragement throughout this demanding journey.

To all of them, we offer our sincere thanks.

Contents

List of Figures	i
List of Tables	iii
General Introduction	1
1 Fundamental Concepts	4
1.1 Introduction	4
1.2 Cloud Computing	4
1.2.1 Definition	4
1.2.2 Characteristics of Cloud Computing	4
1.2.3 Service Models	5
1.2.4 Deployment Models	6
1.3 Edge Computing	6
1.3.1 Definition of Edge Computing	6
1.3.2 Multi-access Edge Computing (MEC)	6
1.3.3 Advantages of Edge Computing	7
1.3.4 Key Applications of Edge Computing	7
1.4 Fog Computing	8
1.4.1 Definition of Fog Computing	8
1.4.2 Essential Characteristics of Fog Computing	8
1.4.3 Architecture of Fog Computing	9
1.5 Cloud, Fog, and Edge Computing: Key Differences	10
1.6 Conclusion	10
2 Distributed Systems	11
2.1 Introduction	11
2.2 Big Data as a Driver	11
2.2.1 Big Data Definition and Context	11
2.2.2 The 5 V's of Big Data	11
2.2.3 Big Data Processing Frameworks	12
2.2.4 Challenges of Big Data	13
2.3 Distributed Systems	13
2.3.1 Definition of a Distributed System	13
2.3.2 System Architectures	14
2.3.3 Distributed Computing Use Cases	15
2.3.4 Role of Distributed Systems in Big Data	15
2.4 Distributed Storage System	16

2.4.1	Distributed object storage system	16
2.4.2	Distributed File System (DFS)	17
2.4.3	Block-Level Distributed Storage System	18
2.5	Challenges in Edge and Mobile Environments	18
2.6	Conclusion	19
3	State of the Art and Review of Existing Solutions	20
3.1	Introduction	20
3.2	Big Data Storage Systems	20
3.2.1	Hadoop Distributed File System (HDFS)	20
3.2.2	Ceph	20
3.2.3	Amazon S3	21
3.3	Peer-to-Peer and Decentralized Storage Systems	21
3.3.1	Kademlia	21
3.3.2	Chord	22
3.3.3	IPFS (InterPlanetary File System)	22
3.4	Mobile and Edge Storage Prototypes	22
3.4.1	Decentralized Storage for Handheld Devices	22
3.4.2	R-Drive: Resilient Storage for Mobile Edge Computing	23
3.4.3	MDFS: Secure Mobile Distributed File System	23
3.4.4	DistressNet-NG: Resilient Storage for Tactical Edge Computing	23
3.4.5	Design and Development of a Storage System in Fog Networking	24
3.5	Enabling Technologies for Next-Generation Mobile Storage	25
3.5.1	Conflict-Free Replicated Data Types (CRDTs)	25
3.5.2	AI-Driven Predictive Resilience	25
3.6	Identification of Research Gaps	26
3.7	Conclusion	27
4	Proposed Solution Design	28
4.1	Introduction	28
4.2	System Overview	28
4.3	Global Architecture	29
4.3.1	Node Types	29
4.3.2	Network Topology	30
4.3.3	Layered Software Architecture	31
4.4	Core Algorithms & Design Decisions	32
4.4.1	Node Discovery & Cluster Formation	32
4.4.2	Cluster head Election	34
4.4.3	Failure Detection	37
4.4.4	Erasur e Coding	38
4.4.5	Fragment Placement and Self-Healing	39
4.4.6	Distributed Catalog	41

4.4.7	Catalog Data Structure	42
4.4.8	Gossip Protocol and Bloom Filter Delta-Sync	42
4.4.9	Security Pipeline	44
4.5	System Requirements	45
4.5.1	Actors	45
4.5.2	User Requirements	45
4.5.3	Administrator Requirements	46
4.6	Use Case Diagrams	46
4.6.1	General Use Case Diagram	47
4.6.2	Administrator System	48
4.7	Class Diagram	48
4.8	Sequence Diagrams	49
4.8.1	Soft Delete File	49
4.8.2	Permanently Delete File	50
4.8.3	File Distribution	51
4.8.4	File Retrieval	52
4.9	Activity Diagram	53
4.10	State Machine Diagram	53
5	Implementation	55
5.1	Introduction	55
5.2	Development Methodology	55
5.3	Development Environment	56
5.3.1	Android Studio	56
5.3.2	Kotlin	57
5.3.3	Jetpack Compose	57
5.3.4	Hilt	58
5.3.5	Room Database	58
5.3.6	Ktor	58
5.3.7	Android NDK and C++	59
5.3.8	React	60
5.4	Core Modules Implementation	60
5.4.1	Node Identity	60
5.4.2	Fragment Encryption	61
5.4.3	Key Distribution	61
5.5	Testing	62
5.5.1	Hardware Setup	62
5.5.2	Network Topology	63
5.5.3	Performance Evaluation by Simulation	63
5.5.4	Real Tests	65
5.6	Application Overview	66
5.6.1	Tab 1: Dashboard	66

5.6.2	Tab 2: Explorer	67
5.6.3	Tab 3: Community	68
5.6.4	Tab 4: Settings	69
5.6.5	Administration Dashboard	70
5.7	Conclusion	71
General Conclusion		72
Bibliography		75
A Detailed Use Case Diagrams		A
B Detailed Scenario Descriptions		D
C Relay Server Administration Dashboard		G

List of Figures

1.1	The 3 service models of Cloud Computing.	5
1.2	Edge computing architecture for cloud gaming.	8
1.3	Three-layer architecture of fog computing.	9
3.1	Architecture of the Storage System in Fog Networking.	24
4.1	Network topology of the system.	30
4.2	Layered Software Architecture	31
4.3	General Use Case Diagram of the System.	47
4.4	Use Case Diagram of Administrator System.	48
4.5	Class Diagram of the System.	48
4.6	Sequence Diagram of Soft Delete File.	49
4.7	Sequence Diagram of Permanently Delete File.	50
4.8	Sequence Diagram of File Storage	51
4.9	Sequence Diagram of the File Retrieval.	52
4.10	Activity Diagram of Cluster Join.	53
4.11	State Machine Diagram of the Node.	54
5.1	Agile iterative development cycle.	56
5.2	Android Studio	56
5.3	Kotlin	57
5.4	Jetpack Compose	57
5.5	Room Database	58
5.6	Ktor	58
5.7	Android NDK and C++	59
5.8	React	60
5.9	Physical network topology of the experimental test cluster and monitoring in- frastructure.	63
5.10	DHT gossip convergence curves (CRDTLWW) for $N \in \{100, 500, 1000, 5000, 10000\}$ nodes with fan-out = 2. The y-axis shows the mean DHT size per node (out of 100 blocks).	64
5.11	Bully re-election after Cluster-head failure at cycle 20 ($N = 1000$ nodes). The network recovers a new stable Cluster-head set within one election cycle (2 simulated seconds).	65
5.12	DHT convergence robustness under mobile churn (0%, 10%, 20%, 30%) for $N = 1000$ nodes with fan-out = 2. The dashed line marks the 95% operational threshold.	65

5.13	Dashboard tab: node role, reliability score, storage contribution, and connectivity health banner.	67
5.14	Explorer.	68
5.15	Community tab: real-time member registry, cluster topology, and heartbeat status.	69
5.16	Settings storage quota configuration.	70
A.1	Use Case Diagram of Authentication and Account Management.	A
A.2	Use Case Diagram of Manage Files.	A
A.3	Use Case Diagram of Manage Folders.	B
A.4	Use Case Diagram of Recycle Bin.	B
A.5	Use Case Diagram of Manage Settings & Preferences.	C
A.6	Use Case Diagram of Administrator System.	C
C.1	Cluster tooltip.	G
C.2	Dashboard.	H
C.3	Node detail panel.	I
C.4	Fragment Map.	I
C.5	Real-time log panel.	J

List of Tables

1.1	Comparison between Cloud, Edge, and Fog Computing.	10
2.1	Advantages and Disadvantages of Distributed Storage Systems	16
3.1	Comparative analysis of decentralized and mobile edge storage solutions. . . .	25
3.2	Identified research gaps in distributed mobile storage systems.	26
4.1	Principal transitions of the node lifecycle finite state machine.	34
4.2	Summary of the leader election protocol design decisions.	36
4.3	Failure scenarios, detection mechanisms, and reactions.	38
4.4	Comparison of redundancy strategies.	38
4.5	Adaptive redundancy profiles as a function of cluster reliability.	39
4.6	Fields of the CatalogEntry structure and their roles.	42
5.1	Hardware specifications of the test cluster devices.	62
5.2	DHT gossip convergence time as a function of network size (fan-out = 2, 100 blocks).	64
5.3	Summary of test results.	66
B.1	Textual description of the File Distribution scenario.	E
B.2	Textual description of the File Retrieval scenario.	F

General Introduction

General Context

We live in an era of pervasive digitalization, in which the volume of data produced worldwide is growing exponentially. The proliferation of smartphones, connected objects (IoT), and mobile applications has turned every user into a permanent producer of data: photos, videos, documents, backups, and personal files accumulate on a daily basis. This phenomenon, commonly referred to as *Big Data*, is characterized not only by the sheer volume of data, but also by its velocity, its variety, and the reliability it demands.

In this context, the need to store, protect, and access one's data at any time has become a fundamental requirement for the modern user. Storage is no longer a mere convenience: it is a central concern of availability, security, and sovereignty over one's own information.

The Cloud and Its Limitations

To meet this demand, *Cloud Computing* has established itself as the dominant solution. It offers elastic storage capacity, high availability, and ubiquitous access to data by relying on massive centralized infrastructures. Public cloud providers thus offer a convenient model in which the user fully delegates the management of their data to a third-party operator.

This centralized model, however, presents several significant limitations. First, **cost**: beyond an often restricted free quota, cloud storage quickly becomes expensive and relies on a recurring subscription model. Second, **dependence on a central authority**: the user loses real control over their data, which is hosted on remote servers governed by commercial, legal, and privacy policies they do not master. Third, **privacy and data sovereignty**: entrusting sensitive data to a single operator exposes the user to risks of surveillance, leakage, or commercial exploitation. Finally, this model assumes **permanent connectivity** to often distant data centers, a source of latency and bandwidth consumption.

Paradoxically, while users pay to store their data in the cloud, they collectively own, through their own smartphones, a considerable and largely underused storage capacity.

Problem Statement

The observation is therefore the following: users have a **growing need for reliable storage**, yet existing cloud solutions are **costly, centralized, and offer little respect for data sovereignty**. At the same time, the distributed storage capacity available on mobile devices remains untapped.

The central question motivating this work is thus: **is it possible to design a distributed, reliable, and secure storage system relying exclusively on a set of ordinary smartphones, without depending on a centralized cloud infrastructure?**

This problem in turn raises several challenges specific to the mobile environment: node volatility (frequent disconnections, or *churn*), limited resources (battery, capacity, computation), device heterogeneity, and the need to guarantee the confidentiality of data stored on devices belonging to third parties.

Proposed Solution and Objectives

To address this problem, we propose a distributed storage system that transforms a federation of ordinary smartphones into a collaborative, resilient, and privacy-preserving storage network. Rather than entrusting data to a central operator, our system distributes each file, in an encrypted and fragmented form, across several participating devices.

The main objectives of the solution are:

- **Decentralization:** eliminate dependence on a centralized cloud infrastructure by relying on the resources of the mobile devices themselves.
- **Resilience:** guarantee continuous data availability despite frequent disconnections, through adaptive Reed-Solomon erasure coding, which is more efficient than simple replication.
- **Security and privacy:** ensure *zero-knowledge* storage, where data is encrypted before leaving the user's device, so that no hosting node can access its content.
- **Adaptability:** dynamically adjust the redundancy level and coordination according to network conditions.
- **Transparency:** hide the complexity of the distributed system behind a simple, easy-to-use mobile application.

Thesis Organization

This thesis is organized into five chapters:

Chapter 1 introduces the fundamental concepts of distributed computing paradigms — *Cloud*, *Edge*, and *Fog Computing* — and establishes the conceptual framework of the work.

Chapter 2 presents the foundations of distributed systems and Big Data, detailing distributed storage architectures and systems, as well as their challenges in mobile environments.

Chapter 3 provides a state-of-the-art review: it analyzes existing storage solutions, whether centralized, peer-to-peer, or specifically mobile, and identifies the research gaps that motivate our work.

Chapter 4 details the design of the proposed solution: the global architecture, the core algorithms (discovery, election, coding, placement, synchronization, security), and the UML modeling of the system.

Chapter 5 describes the implementation and deployment of the application, the technologies employed, the security mechanisms, and presents the results of the tests conducted on a real cluster of smartphones.

Finally, a **general conclusion** summarizes the work carried out and opens perspectives for future improvement.

1 Fundamental Concepts

1.1 Introduction

In today's digital era, where speed and efficiency are paramount, traditional centralized computing architectures often fall short in meeting the demands of real-time, data-intensive applications. This chapter introduces the fundamental concepts of distributed computing paradigms, namely Cloud, Fog, and Edge Computing. It outlines their definitions, core characteristics, and architectural designs, and highlights their crucial roles in enhancing data processing and supporting real-time applications. Furthermore, a comparative analysis is presented to underscore the key differences among these paradigms and their most suitable use cases.

1.2 Cloud Computing

1.2.1 Definition

According to the National Institute of Standards and Technology (NIST), Cloud Computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources, such as networks, servers, storage, applications, and services, that can be rapidly provisioned and released with minimal management effort or service provider interaction. [1]

This cloud model is composed of five essential characteristics, three service models, and four deployment models.

1.2.2 Characteristics of Cloud Computing

Cloud computing is defined by five essential characteristics. These are described below: [1]

- **On-demand self-service:** A consumer can unilaterally provision computing capabilities, such as server time and network storage, automatically as needed, without requiring human interaction with each service provider.
- **Broad network access:** Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, tablets, laptops, and workstations).
- **Resource pooling:** The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. Customers generally have no control or knowledge over the exact location of the provided resources, but

may specify location at a higher level of abstraction (e.g., country, state, or datacenter). Examples of resources include storage, processing, memory, and network bandwidth.

- **Rapid elasticity:** Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be appropriated in any quantity at any time.
- **Measured service:** Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service.

1.2.3 Service Models

There are typically 3 types of service models in the Cloud. They are described below: [1]

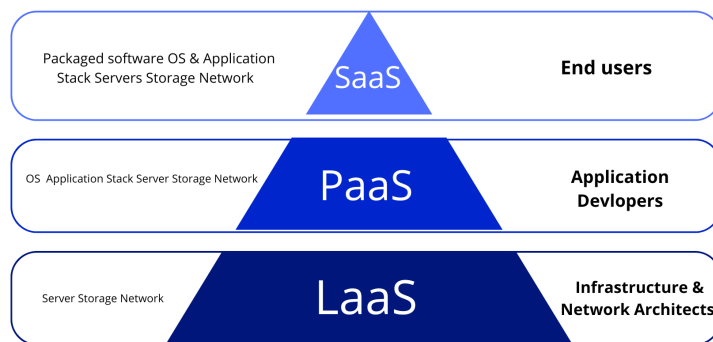


Figure 1.1: The 3 service models of Cloud Computing.

- **Software as a Service (SaaS):** In SaaS, consumers have to purchase the ability to access or use an application or service hosted in the cloud.
- **Platform as a Service (PaaS):** PaaS allows consumers to purchase access to platforms, allowing them to deploy their software and applications on the cloud. The consumer does not manage the operating systems or the network access, which can create some constraints on the nature of applications that can be deployed. Amazon Web Services, Rackspace, and Microsoft Azure are examples.
- **Infrastructure as a Service (IaaS):** Here, consumers can control and manage the operating systems, applications, network connectivity, and storage, without controlling the cloud themselves.

1.2.4 Deployment Models

Cloud computing supports four deployment models, each defined by the scope of access and ownership of the infrastructure [1]:

- **Public Cloud:** The cloud infrastructure is provisioned for open use by the general public. It is owned and operated by a cloud service provider and exists on the provider's premises (e.g., Amazon Web Services, Google Cloud, Microsoft Azure).
- **Private Cloud:** The cloud infrastructure is provisioned for exclusive use by a single organization. It may be owned and managed by the organization itself or by a third party, and may exist on or off premises.
- **Hybrid Cloud:** The cloud infrastructure is a composition of two or more distinct cloud models (public, private, or community) that remain separate entities but are bound together by standardized technology enabling data and application portability.
- **Community Cloud:** The cloud infrastructure is provisioned for exclusive use by a specific community of consumers from organizations that share common concerns (e.g., security requirements, compliance, or mission). It may be owned and managed by one or more of the community organizations or by a third party.

1.3 Edge Computing

1.3.1 Definition of Edge Computing

The rapid growth of data and the increasing number of connected devices have exposed several limitations of traditional cloud computing, including high latency, network bandwidth overload, high energy consumption, and potential security and privacy risks. To address these challenges, edge computing has emerged as an effective solution.

Based on the definition from CISCO, "Edge computing is a decentralized IT model that handles data processing near its point of origin through local computing, storage, networking, and security resources. By operating closer to where data is produced, it minimizes latency, enhances real-time responsiveness, and reduces bandwidth consumption. This approach is particularly critical for applications that require immediate decision-making, such as industrial automation, smart retail systems, and telemedicine services." [2]

1.3.2 Multi-access Edge Computing (MEC)

Multi-access Edge Computing (MEC) is a network architecture that extends cloud computing capabilities to the edge of the network, where data is generated and consumed. Rather than routing all traffic to distant cloud servers, MEC processes data locally at edge nodes such as base stations and access points, achieving ultra-low latency in the single-digit millisecond range. The

multi-access aspect distinguishes MEC from general edge computing by supporting not only mobile networks but also Wi-Fi and fixed broadband, making it applicable across heterogeneous access technologies. [3]

1.3.3 Advantages of Edge Computing

Edge computing offers several advantages over traditional centralized architectures, particularly for applications that generate large volumes of data and require immediate processing. By distributing computational resources closer to the data source, it addresses the key limitations of cloud-only models. [4]

0. **Reduced latency:** By processing data closer to the source, edge computing minimizes the time it takes for data to travel to a centralized cloud server and back. This is crucial for applications that require real-time or near-real-time responses, such as autonomous vehicles, industrial automation, and augmented reality.
0. **Bandwidth optimization:** Edge computing reduces the amount of data that needs to be transmitted to centralized cloud servers, thereby optimizing bandwidth usage. This is particularly important in scenarios where large volumes of data are generated, such as in IoT deployments.
0. **Enhanced security and privacy:** By processing sensitive data locally, edge computing can help mitigate security risks associated with transmitting data over long distances. This is especially relevant for applications that handle personal or confidential information.
0. **Scalability:** Edge computing allows for the distribution of computational resources across multiple edge nodes, enabling applications to scale more effectively as demand increases.

1.3.4 Key Applications of Edge Computing

- **Autonomous vehicles:** Self-driving cars and trucks continuously collect and analyze data from multiple sensors, including radar, LiDAR, and cameras. Edge computing enables real-time processing of this data locally, allowing the navigation system to respond instantly to changing traffic conditions without relying on distant cloud servers.
- **Healthcare:** In hospitals, the speed of information processing can be critical. Edge computing enables real-time analysis of patient data locally, allowing doctors to access live dashboards and supporting complex interventions such as robot-assisted surgeries where millisecond delays are unacceptable.
- **Cloud gaming:** Cloud gaming streams live game feeds directly to devices while processing the game remotely. Edge servers deployed close to gamers minimize latency, ensuring responsive and immersive gameplay.

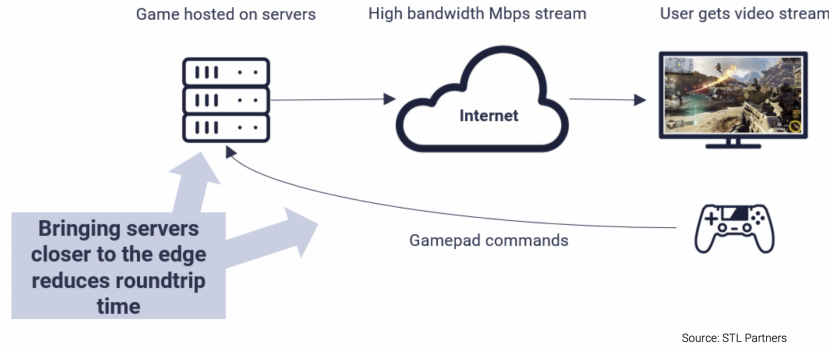


Figure 1.2: Edge computing architecture for cloud gaming.

1.4 Fog Computing

1.4.1 Definition of Fog Computing

According to NIST, fog computing is a layered model for enabling ubiquitous access to a shared continuum of scalable computing resources [5]. It facilitates the deployment of distributed, latency-aware applications and services through fog nodes (physical or virtual) residing between smart end-devices and centralized cloud services. These fog nodes are context-aware and can be organized in clusters, either vertically to support isolation, or horizontally to support federation. By providing local computing resources close to end-devices, fog computing minimizes request-response time while maintaining network connectivity to centralized services when needed.

In the field of data storage and processing and the IoT, fog computing creates an intermediate layer positioned between edge computing and cloud computing, combining the proximity of edge processing with the scalability of cloud infrastructure.

Fog Node

The fog node is the core component of the fog computing architecture. It can be a physical device (gateway, switch, router, server) or a virtual component (virtual machine, cloudlet) positioned close to end-devices or access networks. Each fog node is aware of its geographical and logical location within its cluster, and provides computing, storage, and communication services between the network edge and the cloud. Fog nodes can operate in standalone mode or be federated into clusters for horizontal scalability across distributed locations. [5]

1.4.2 Essential Characteristics of Fog Computing

NIST identifies six characteristics that distinguish fog computing from other paradigms [5]:

- **Contextual location awareness and low latency:** Fog nodes are aware of their logical position in the network and are co-located with end-devices, enabling the lowest possible response time compared to centralized cloud services.

- **Geographical distribution:** Unlike centralized cloud deployments, fog services are distributed across identifiable geographic locations to serve mobile and location-dependent applications.
- **Heterogeneity:** Fog computing supports data collection and processing across different device types and network communication protocols.
- **Interoperability and federation:** Fog components from different providers can interoperate, and services can be federated across domains to support seamless delivery.
- **Real-time interactions:** Fog applications are designed for real-time processing rather than batch operations.
- **Scalability and agility:** Fog node clusters are adaptive, supporting elastic compute, resource pooling, and dynamic adjustments to data-load and network condition variations.

1.4.3 Architecture of Fog Computing

The fog computing architecture is generally composed of three different working layers, namely terminal layer, fog layer and cloud layer. The three-layer architecture of fog computing and its detailed description are shown in Figure 1.3.

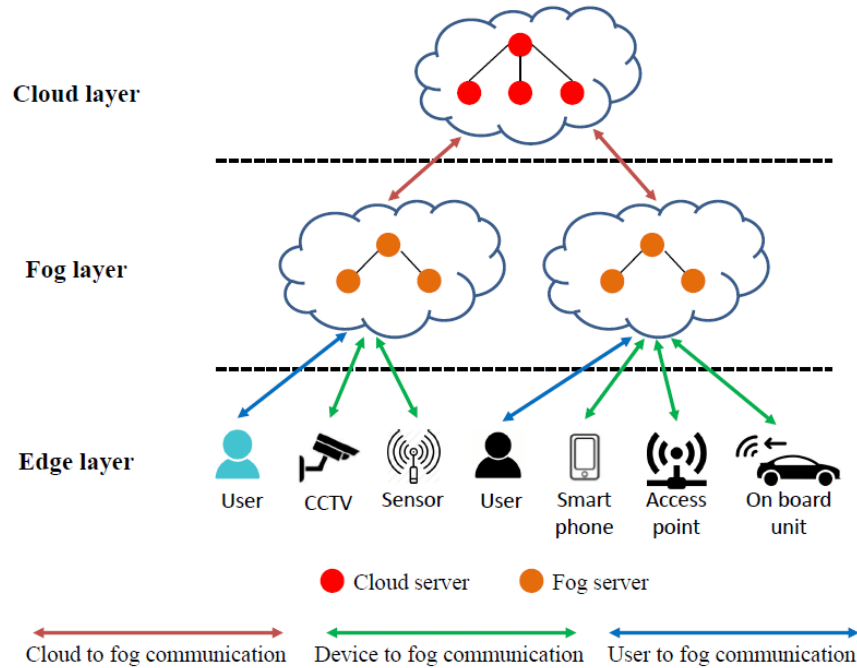


Figure 1.3: Three-layer architecture of fog computing.

- **Edge layer:** The edge layer is the closest to end users and end devices and includes various IoT and intelligent devices such as sensors, smartphones, smart vehicles, smart cards, and readers. Although these devices possess limited computing capabilities, they are mainly

used for data sensing and collection. The sensed data are then transmitted to upper layers for further processing and storage.

- **Fog layer:** Located at the network edge, this layer consists of numerous fog nodes (routers, gateways, access points, base stations, fog servers) that can be fixed or mobile and are distributed between end devices and the cloud (e.g., cafes, streets, shopping centers). They process, transmit, and temporarily store data, enabling real-time analysis and delay-sensitive applications. Connected to IP networks and cloud data centers, fog nodes can also leverage more powerful computing and storage through collaboration with the cloud.
- **Cloud layer:** This layer consists of multiple high-performance servers and storage devices that support intelligent applications such as smart homes, transportation, factories, and healthcare. It provides powerful computing and storage for large-scale data processing and analysis. Unlike traditional cloud computing, fog computing does not rely entirely on the cloud; control strategies are applied to manage and schedule cloud resources efficiently, improving utilization.

1.5 Cloud, Fog, and Edge Computing: Key Differences

Table 1.1 provides a comparative overview of Cloud, Fog, and Edge Computing, highlighting their unique features and applications.

	Cloud Computing	Edge Computing	Fog Computing
Latency	High	Low / Medium	Low / Medium
Scalability	High	Yes	Yes
Network Architecture	Centralized	Distributed	Distributed
Availability	High (99.99%)	Device-dependent	Highly redundant
Mobility	Limited	Supported	Supported
Storage Capabilities	Strong	Weak	Medium
Bandwidth Cost	High	Low	Low

Tableau 1.1: Comparison between Cloud, Edge, and Fog Computing.

1.6 Conclusion

This chapter introduced the computing paradigms of Cloud, Fog, and Edge. Cloud provides scalable centralized resources, Fog brings computation closer to data sources, and Edge processes data directly at the source for real-time applications. Understanding these paradigms provides the necessary context for the following chapter, which explores fully decentralized peer-to-peer architectures that form the foundation of the proposed system.

2 Distributed Systems

2.1 Introduction

This chapter reviews the core concepts of Big Data and distributed storage, presents major distributed storage technologies, compares key solutions in terms of performance and scalability, and discusses their limitations, especially for edge and mobile environments. The goal is to build a solid foundation for the research work presented later in this thesis.

2.2 Big Data as a Driver

2.2.1 Big Data Definition and Context

Big Data refers to extremely large and complex datasets that cannot be efficiently processed or analyzed using traditional database systems. These datasets require specialized storage, distributed processing frameworks, and advanced analytical techniques to extract meaningful insights and support decision-making [6].

2.2.2 The 5 V's of Big Data

A dataset is not considered Big Data solely because of its size. To qualify as Big Data, it must exhibit certain key characteristics, commonly referred to as the **5 V's**. These characteristics capture the main challenges in storing, processing, and analyzing modern large-scale datasets [7].

- **Volume:** Big Data involves extremely large datasets that require reliable storage and advanced processing techniques to manage and analyze effectively.
- **Velocity:** Data can now be processed and analyzed as it is generated, enabling real-time decision-making and rapid responses to opportunities or threats.
- **Variety:** Big Data includes a mix of structured, semi-structured, and unstructured data from diverse sources, which traditional databases cannot easily handle.
- **Veracity:** The usefulness of Big Data depends on the accuracy, relevance, and trustworthiness of the data, which can be affected by errors, biases, or inconsistent sources.
- **Value:** The ultimate goal of Big Data is to generate meaningful insights that deliver measurable business benefits and support operational resilience.

2.2.3 Big Data Processing Frameworks

Big Data storage alone is not sufficient to extract insights from massive datasets. Efficient processing requires **distributed frameworks** capable of handling large volumes, high velocity, and heterogeneous data. Among the most widely used frameworks are **Hadoop** and **Spark**.

2.2.3.1 Hadoop

Apache Hadoop is an open-source platform for distributed storage and processing of very large datasets. It enables parallel computation and fault-tolerant storage, suitable for large-scale batch analytics [8]. Its core components are:

- **HDFS (Hadoop Distributed File System):**
 - Stores data across multiple nodes by splitting it into blocks.
 - Replicates blocks to provide fault tolerance.
 - Ensures high-throughput access for very large datasets.
- **MapReduce:**
 - Parallel processing framework for large datasets.
 - **Map phase:** processes and filters input data into key-value pairs.
 - **Reduce phase:** aggregates results from the Map phase.
 - Provides automatic parallelization and fault tolerance by rerunning failed tasks.
- **YARN (Yet Another Resource Negotiator):**
 - Manages cluster resources (CPU, memory, etc.) across multiple applications.
 - Enables multiple frameworks to run concurrently on the same cluster.
 - Optimizes cluster utilization and improves overall system efficiency.

2.2.3.2 Apache Spark

Apache Spark is an open-source distributed processing system designed for fast analytics, both batch and real-time. Key features include in-memory processing and support for multiple workload types [9]:

- **Spark Core:**
 - Foundation of Spark; handles task scheduling, memory management, and fault tolerance.
 - Uses **Resilient Distributed Datasets (RDDs)** for fault-tolerant, parallel data processing.

- **In-memory Processing:**

- Stores intermediate results in memory, drastically reducing read/write time to disk.
- Improves performance for iterative algorithms, such as machine learning and graph processing.

- **Higher-level Libraries:**

- **Spark SQL:** structured data processing.
- **Spark Streaming:** real-time data processing.
- **MLlib:** machine learning library.
- **GraphX:** graph processing engine.

2.2.4 Challenges of Big Data

Big Data presents several challenges, particularly related to **data storage**, despite its promising opportunities:

- **Data Volume:** - The sheer **volume of data** is a major challenge. - Despite advances in **data storage technologies**, data volumes approximately double every two years. - Companies still face difficulties in managing this growth and in finding **efficient storage solutions**.
- **Data Preparation:** - Simply storing data is not enough. - For data to be useful, it must be **organized and processed** beforehand. - **Data cleaning** and ensuring its relevance for analysis require considerable effort. - Data experts spend between 50 and 80% of their time on **data organization and preparation** before it can be effectively used.
- **Evolving Technologies:** - Big Data technologies are evolving rapidly. - A few years ago, **Apache Hadoop** was the most widely used technology for Big Data processing. - Later, **Apache Spark** appeared in 2014. - Today, combining these two infrastructures seems to be the most promising approach. - Mastering Big Data technology remains a **continuous challenge**.

2.3 Distributed Systems

2.3.1 Definition of a Distributed System

A **distributed system** is a collection of independent computers (or nodes) that **appear to users as a single coherent system**. These nodes communicate and coordinate their actions through a network to achieve common goals, such as **data storage, computation, or service delivery**, while **hiding the complexities of the underlying distribution** [10, 11].

Distributed systems are characterized by **concurrency, lack of a global clock, and fault tolerance**, enabling them to provide **scalable, reliable, and efficient services** even in the presence of node failures or network issues.

2.3.2 System Architectures

While architectural styles describe how software components are logically organized, *system architectures* describe how these components are physically placed on machines and which entity controls coordination. This is the level at which distributed systems are usually classified into three main models [10, 11].

2.3.2.1 Centralized Architecture (Client–Server)

In the centralized model, the system is organized around the client–server paradigm: clients request services and servers provide them, resulting in an inherently asymmetric relationship [12]. This model is commonly structured into multiple tiers — two-tier, three-tier, or more generally n -tier architectures — where presentation, application logic, and data management are separated across distinct levels. Typical examples include the World Wide Web (web browsers querying web servers), email systems based on SMTP/IMAP servers, and traditional relational database management systems accessed by remote clients. Although simple to design and administer, centralized architectures suffer from a single point of failure and limited scalability, as the server constitutes a potential bottleneck.

2.3.2.2 Decentralized Architecture (Peer-to-Peer)

In the decentralized, or Peer-to-Peer (P2P), model, all nodes are equal and each peer acts simultaneously as a client and a server, sharing resources directly without any central coordinating entity [13]. P2P systems are generally classified into three categories:

- **Structured P2P:** peers and data are organized according to a deterministic structure, typically a Distributed Hash Table (DHT), which guarantees efficient lookup in $O(\log N)$ hops. Representative examples include Chord [14] and Kademlia [15], the latter being used in widely deployed systems such as the BitTorrent DHT and IPFS.
- **Unstructured P2P:** peers are connected following a random or ad hoc topology, with no predefined organization of the data. Resource discovery typically relies on flooding or random walks. Early file-sharing systems such as Gnutella and the original Napster overlay are classic examples.
- **Hybrid P2P (Super-Peers):** a subset of more capable nodes, called *super-peers*, take on additional coordination responsibilities on behalf of regular peers [16], combining the resilience of pure P2P with the efficiency of partial centralization. This design was popularized by file-sharing systems such as KaZaA and later versions of Gnutella.

2.3.2.3 Hybrid Architecture

The hybrid model combines centralized and decentralized elements to balance their respective advantages. A common design employs a minimal centralized component for bootstrapping and node discovery, while data exchange and coordination remain decentralized across the participating nodes [16]. The original Napster is a canonical example: it relied on a central server for indexing and peer lookup, while the actual file transfers occurred directly between peers. Modern messaging and voice-over-IP systems, as well as many super-peer overlays, follow the same principle. This approach preserves the simplicity of centralized discovery without sacrificing the resilience and scalability of a decentralized data plane, which explains its frequent adoption in large-scale and mobile distributed systems.

2.3.3 Distributed Computing Use Cases

Distributed computing is widely applied across industries and platforms to enable scalability, fault tolerance, and efficient processing of large workloads. Key application domains include:

- **Communications:** The communications industry extensively relies on distributed computing. Telecommunication networks, whether traditional telephone networks or modern cellular systems, function as distributed systems. The internet and e-mail are major examples of communication-based distributed computing [17].
- **Computing and Artificial Intelligence:** Computing is increasingly dominated by advances in Artificial Intelligence (AI) and Machine Learning (ML). These technologies require massive volumes of data and significant processing power. Distributed computing enables training of large-scale neural networks, supports federated learning, and provides the necessary parallel processing to handle complex AI/ML workloads [17].
- **Data Management:** Distributed computing transforms complex data management and storage tasks into subtasks distributed across nodes. Distributed databases, data centers, and cloud storage platforms rely on distributed architectures to improve performance, availability, and scalability [17].
- **Medical and Healthcare:** Distributed computing enables advanced technologies such as robotic surgery, pharmaceutical design, and medical simulations. It provides the computation and graphics resources required for detailed 3D modeling and analysis [17].

2.3.4 Role of Distributed Systems in Big Data

Distributed systems play a central role in Big Data environments by providing the mechanisms required to manage very large volumes of data in a **scalable**, reliable, and efficient manner. They enable **data storage and processing** to be distributed across multiple nodes, allowing for parallel processing and fault tolerance.

In the context of Big Data, distributed systems ensure:

- **Horizontal scalability:** Allowing new resources to be added without interrupting service.
- **Fault tolerance:** Through mechanisms such as data replication.
- **Efficient processing:** Via parallel execution of tasks across distributed clusters.
- **Data locality:** Bringing computation closer to data to reduce communication costs.

Big Data frameworks such as **Hadoop** and **Apache Spark** rely directly on fundamental distributed system principles to coordinate nodes, manage resources, and ensure data consistency [10, 18, 19].

2.4 Distributed Storage System

Definition A distributed storage system is a data storage architecture that utilizes multiple servers to cooperatively store data. It divides information into multiple pieces and distributes them across different servers to ensure availability and efficiently manage large-scale data. The primary motivations for adopting distributed storage systems include the need for scalability, fault tolerance, and enhanced performance.

The following table summarizes the key advantages and disadvantages of distributed storage systems:

Tableau 2.1: Advantages and Disadvantages of Distributed Storage Systems

Advantages	Disadvantages
Scalability	Complexity
Fault Tolerance	Consistency issues
High Availability	Network Overhead
Performance	Security challenges
Cost-Effectiveness	—

A distributed storage system can relate to one of three storage types: block, file, and object; these are defined below:

2.4.1 Distributed object storage system

Object Storage enables the management of vast volumes of unstructured data. The system is based on objects and metadata, where each file corresponds to an object associated with an access path (address) and a unique identifier. These objects are stored in a flat address space, ensuring the system remains highly scalable. [20] An object storage system must satisfy several key properties to guarantee a high Quality of Service (QoS). These properties are defined as follows:

-
- **Connectivity:** The storage system must ensure operational continuity even when a site is disconnected. This resilience allows users to maintain access to required data even if the geographically closest site becomes unavailable [21]
 - **Data Locality:** This is the primary motivation behind the Fog computing paradigm. The objective is to prioritize local access whenever possible; thus, users should ideally perform write operations on the nearest site to minimize data access latency [22]
 - **Inter-site Network Isolation:** Actions such as reading or writing performed at one site must not negatively impact the performance of other sites. In other words, if a specific site experiences a sudden spike in activity, the performance of the remaining sites in the network should remain stable [23].
 - **Mobility Support:** Data must transparently "follow" the users as they move. The storage service should be capable of migrating requested data from a previous site to a new one seamlessly. Such transparent data relocation is essential to minimize remote access overhead and maintain low latency [22].
 - **Scalability:** The system must be designed to scale efficiently, supporting a vast number of Fog sites, a high volume of concurrent clients per site, and a massive number of stored objects without performance degradation.

2.4.2 Distributed File System (DFS)

Definition A Distributed File System (DFS) is a system designed to manage files and folders across multiple networked computers [24]. In this model, data is physically stored on one or more servers but is accessed and processed as if it were stored on the local client machine. Despite this local appearance, the servers maintain full control over the data and provide centralized access control for all clients [25].

A DFS facilitates the sharing of information and files among network users in a controlled and authorized manner. This information is shared through a **unified hierarchical view**, allowing users to navigate a single directory structure regardless of the physical location of the files. To ensure high performance and reliability, a distributed file system must be built upon three core principles:

[26]

Transparency: The system should hide the details of data distribution and location, making the network's complexity invisible to the end-user.

Fault Tolerance: The system must continue to operate correctly even in the event of hardware or software failures, typically achieved through redundancy.

Scalability: The system should maintain its performance and efficiency as the number of users, files, or nodes increases significantly.

2.4.3 Block-Level Distributed Storage System

Definition A block is a sequence of bytes. Block-based storage interfaces are the most common method for storing data on rotating media, such as hard drives, CDs, floppy disks, and even traditional 9-track tapes. The ubiquity of block device interfaces makes a virtual block device an ideal candidate for interacting with mass storage systems [23]. Currently, block-level storage remains one of the most popular storage system technologies [27].

In this storage mode, raw data volumes are created, and each block can be managed as an individual hard drive. These blocks are controlled by server-based operating systems, and each volume can be individually formatted with the required file system.

Specifications According to [27], block-level storage possesses the following characteristics:

- **Versatility:** Block-level storage can be used to store standard files and serves as the primary storage for specialized applications, such as databases and virtual machine file systems (VMFS).
- **Efficiency and Reliability:** The transport mechanisms for block-level data are highly efficient and reliable, minimizing overhead during high-performance operations.
- **Independence:** Each storage volume can be treated as an independent disk drive and can be controlled by an external server operating system.
- **Standardized Protocols:** Block-level storage primarily utilizes protocols such as *iSCSI* and *FCoE* for data transfer. In these environments, *SCSI* commands act as the communication interface between the **initiator** (the client) and the **target** (the storage resource).

2.5 Challenges in Edge and Mobile Environments

While the solutions discussed excel in dedicated, high-performance data centers, they face several critical limitations when deployed in **edge and mobile computing** contexts:

- **Intermittent Connectivity (Churn):** Mobile nodes join and leave the network frequently. Traditional systems like HDFS may interpret a temporary disconnection as a failure, triggering constant and energy-draining data re-replication.
- **Resource Constraints:** Distributed storage daemons require significant memory and CPU. The power consumption of background synchronization tasks is often too high for battery-powered mobile devices.
- **Network Heterogeneity:** Unlike data center networks with uniform low latency, mobile environments face fluctuating bandwidth and high latency, which disrupts synchronous consensus protocols.

-
- **Data Privacy and Security:** Mobile edge nodes are more vulnerable to physical theft or unauthorized access, requiring decentralized encryption and trust models [28].

2.6 Conclusion

This chapter presented the fundamentals of distributed systems and their essential role in Big Data environments. Key distributed storage architectures, frameworks, and solutions were reviewed, highlighting their advantages, limitations, and trade-offs. Understanding these concepts is crucial for designing scalable, fault-tolerant, and high-performance distributed storage and processing systems in large-scale and heterogeneous data contexts.

3 State of the Art and Review of Existing Solutions

3.1 Introduction

The proliferation of mobile devices and the Internet of Things (IoT) has led to an exponential increase in data generated at the network edge, extending Big Data paradigms beyond traditional data centers towards highly distributed, mobile, and volatile environments. Designing storage architectures for such scenarios requires addressing mobility, intermittent connectivity, and energy constraints — challenges that conventional models were not designed to handle.

This chapter critically analyzes current solutions across three axes: (1) centralized Big Data storage systems, (2) peer-to-peer and decentralized approaches, and (3) mobile-specific storage prototypes. Each system is evaluated in terms of scalability, decentralization, fault tolerance, and suitability for mobile environments. The chapter then surveys emerging enabling technologies and concludes with a structured identification of the research gaps that motivate the present work.

3.2 Big Data Storage Systems

3.2.1 Hadoop Distributed File System (HDFS)

HDFS, inspired by the Google File System [29], centralizes metadata on a single NameNode and distributes data blocks across DataNodes, with configurable replication (default 3×) for fault tolerance. It supports petabyte-scale storage on commodity hardware and integrates with MapReduce for batch processing.

Limitations for mobile contexts: The NameNode is a single point of failure, and HDFS requires high-speed wired networks and stable power. Its high sensitivity to churn makes it incompatible with mobile ad hoc environments.

3.2.2 Ceph

Ceph is a leading open-source Software-Defined Storage (SDS) platform that decouples physical hardware from storage management logic, achieving high scalability (up to several petabytes) through policy-based automation. Through its core abstraction layer RADOS (Reliable Autonomous Distributed Object Store), Ceph provides a unified storage solution combining block storage (Ceph Block), native object storage (RADOS), and a POSIX-compliant distributed file system (CephFS) [30]. Its key advantage in Fog computing contexts is the CRUSH algorithm,

which allows clients to compute data locations locally, eliminating the need for a centralized metadata server [31].

Limitations for mobile contexts: CRUSH assumes stable Object Storage Daemons (OSDs), an assumption violated by mobile devices with unpredictable availability. Continuous cluster map synchronization also generates excessive traffic on low-bandwidth mobile links and drains battery.

3.2.3 Amazon S3

Amazon Simple Storage Service (S3) is a globally distributed object storage platform whose design is inspired by Dynamo [32], leveraging consistent hashing, replication, and quorum-based updates for high durability and availability. Its infrastructure spans multiple regions, each composed of several Availability Zones (AZs) hosting fleets of storage servers and microservices. Data is persisted using Log-Structured Merge (LSM) trees with crash consistency via soft updates [33], achieving eleven nines (99.999999999%) of durability through erasure coding, cross-AZ replication, and continuous automatic repair. S3 now provides strong read-after-write consistency [34], and offers multiple storage classes (Standard, Standard-IA, Glacier, Express One Zone) optimized for cost, latency, and durability.

Relevance for mobile distributed storage: Although S3 relies on fixed infrastructure and high-bandwidth networks, it illustrates several principles relevant to mobile storage: efficient erasure-coded redundancy and repair, consistency management for concurrent updates, and automated integrity verification.

3.3 Peer-to-Peer and Decentralized Storage Systems

3.3.1 Kademlia

Kademlia [15] is a Distributed Hash Table (DHT) that uses XOR-based distance metrics for decentralized node and data location with $O(\log N)$ lookup complexity (e.g., ~ 14 hops for 10,000 nodes), achieving resilience to churn through k-bucket maintenance and iterative lookups.

Limitations for mobile contexts: While Kademlia is the standard for decentralized lookup in stable networks, several characteristics make it unsuitable for smartphones. Its continuous background maintenance (frequent Ping and Find_Node messages) prevents the processor from entering deep-sleep states, draining the battery. Because its routing policy favors long-lived nodes, routing tables become obsolete within minutes in volatile mobile networks. The $O(\log N)$ multi-hop lookups consume mobile data and bandwidth, especially when IP addresses change frequently. Finally, Kademlia is both resource-agnostic — placing fragments without regard to a device’s storage or battery state — and topologically blind, assuming a fully connected IP network rather than the physical neighbor constraints of device-to-device (D2D) scenarios.

3.3.2 Chord

Chord [14] organizes nodes and data keys into a **logical ring** using consistent hashing. Each node and item receives an m -bit identifier (via a hash such as SHA-1), and a key k is stored on its *successor*, the first node whose ID equals or follows k . A per-node **Finger Table** enables query resolution in $O(\log N)$ hops (e.g., ~ 10 steps for 1,000 nodes).

Limitations for mobile contexts: Chord’s rigid structure is difficult to maintain in a mobile-first environment. Its periodic stabilization protocol generates constant background traffic that drains the battery, and the precise ring structure is fragile: the simultaneous departure of contiguous successors — common when a group of users moves out of range — can break the ring and cause permanent data loss. As with Kademlia, the logical ring ignores physical proximity, so a single logical hop may span many physical hops in a MANET, making the $O(\log N)$ guarantee far more costly in time and energy than in a wired network.

3.3.3 IPFS (InterPlanetary File System)

IPFS [35] is a content-addressed P2P storage network where objects are identified by cryptographic hashes (CIDs), organized in Merkle DAGs for immutable storage and verification, and routed via a Kademlia-based DHT.

Limitations for mobile contexts: Mobile devices typically act as passive consumers rather than active storage nodes, since running a full IPFS daemon demands continuous CPU, memory, and network resources. Studies further indicate that over 95% of IPFS content is served by a few centralized gateways, undermining its decentralization claims in practice. More broadly, while DHTs and content-addressed networks deliver decentralization and scalability, they are not optimized for the high churn, intermittent connectivity, and energy limitations of mobile devices.

3.4 Mobile and Edge Storage Prototypes

This section reviews the academic prototypes most directly related to the present work — systems that implement distributed storage on mobile devices.

3.4.1 Decentralized Storage for Handheld Devices

Monteiro et al. [36] introduced a fully decentralized storage framework for handheld devices that aggregates their local storage into a peer-to-peer (P2P) mobile cloud. Built on a mobile-optimized DHT (*TomP2P*) with three adaptive replication strategies (static, network-aware, and popularity-based), its key feature is an *Energy Manager* that disables energy-intensive tasks when the battery falls below a critical threshold. The prototype was validated through an Android shared photo gallery application.

Critical analysis: Despite its energy-aware design, the system depends on a central Wi-Fi access point and lacks pure D2D support (e.g., Wi-Fi Direct). Its reliance on simple replication rather than erasure coding provides only “best-effort” persistence, and its evaluation was limited to ~10 devices. The absence of integrated cryptography also leaves stored data exposed.

3.4.2 R-Drive: Resilient Storage for Mobile Edge Computing

R-Drive [37] is a resilient storage framework for Mobile Edge Computing (MEC), targeting disaster response and tactical scenarios without infrastructure. It combines *Reed-Solomon* erasure coding with a layered security approach (*AES-256* encryption and *Shamir’s Secret Sharing* for key fragmentation). Its **RSock** resilient transport layer enables opportunistic transfer through “mobile data mules” that physically carry fragments across disconnected regions, and a distributed directory service provides a hierarchical namespace with UNIX-style access control. A prototype achieved 11.5 MB/s throughput and ~3.5 hours of battery life on standard smartphones.

Critical analysis: R-Drive remains reliant on semi-fixed “Edge nodes” and is therefore not optimized for a cloud of purely mobile smartphones. The absence of a DHT limits lookup scalability, and its energy management remains partial, lacking proactive cluster-wide battery analytics.

3.4.3 MDFS: Secure Mobile Distributed File System

MDFS, developed by Huchton [38] at the Naval Postgraduate School, is one of the foundational efforts to implement a distributed file system natively on Android smartphones for tactical ad-hoc environments. Its security-first architecture combines *AES encryption*, *erasure coding*, and *Shamir’s threshold-based secret sharing*, achieving ~10 Mbps throughput. It remained, however, a proof-of-concept for secure distribution rather than a scalable cloud solution.

Critical analysis: MDFS relies on a flat network topology with no DHT or structured indexing, limiting scalability. It incorporates no energy management, leading to rapid battery depletion given the cost of encryption and erasure coding, and provides no metadata synchronization or consistency management during network partitions. Developed in 2011, it also predates modern Android background constraints, high-speed D2D protocols, and current storage APIs.

3.4.4 DistressNet-NG: Resilient Storage for Tactical Edge Computing

DistressNet-NG [39] extends R-Drive for disaster management and tactical operations, integrating four components: R-Drive (erasure-coded encrypted storage), R-Share (data sharing), EdgeKeeper (resilient coordination), and RSock (communication over degraded links). Deployed on Android devices carried by first responders, it enables local data analytics on nearby devices, minimizing Internet dependency.

Critical analysis: The system is designed exclusively for tactical/military scenarios, its edge nodes remain semi-fixed, and it provides no DHT-based scalable routing, no CRDTs for

post-partition consistency, and no AI-based prediction of node churn.

3.4.5 Design and Development of a Storage System in Fog Networking

Baza and Baza [40], in their 2018 Master's thesis at USTHB (Faculty of Electronics and Computer Science), proposed a distributed storage system based on the *Fog Computing* paradigm, aiming to overcome the high latency and network congestion of centralized cloud storage by bringing storage closer to IoT devices. The architecture aggregates the local storage of interconnected *Fog nodes* into a **shared distributed storage space**, distributing data across nodes for availability and fault tolerance, and exposes a **web-based application** that abstracts the underlying complexity from end users (Figure 3.1).

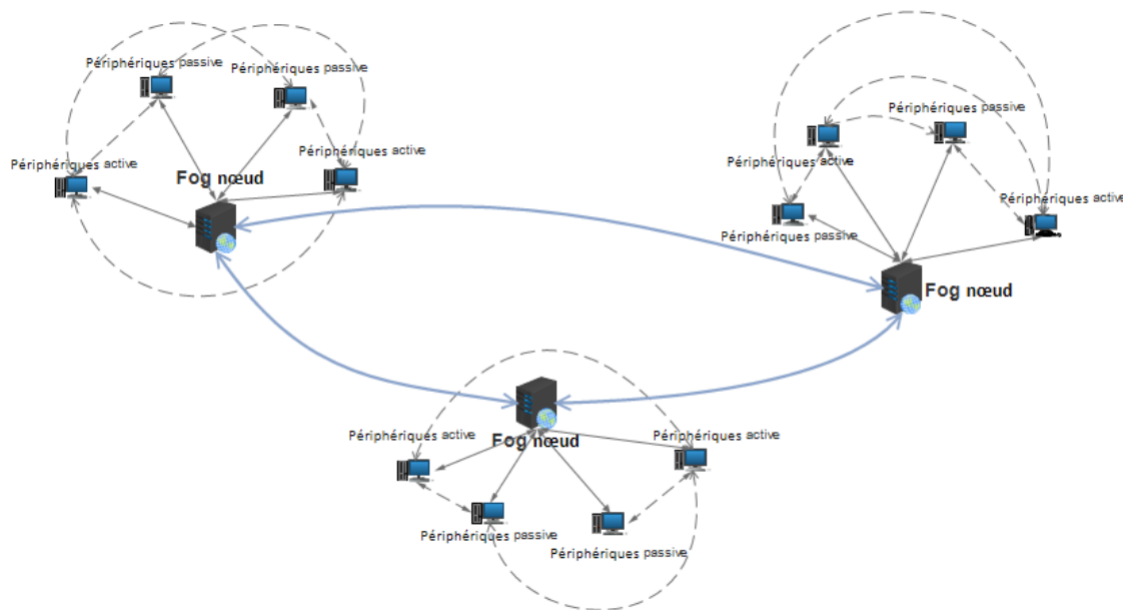


Figure 3.1: Architecture of the Storage System in Fog Networking.

Critical analysis: The system targets a fixed, indoor environment, which limits its use in mobile scenarios. Its reliability depends heavily on the availability and resources of participating nodes, and managing data distribution and synchronization in a Fog environment adds significant complexity in terms of scalability and management.

These prototypes demonstrate practical implementations of mobile edge storage but still face limitations in scalability, energy efficiency, and predictive resilience. Table 3.1 summarizes their comparative analysis, and the next section surveys the enabling technologies that could address these shortcomings.

Tableau 3.1: Comparative analysis of decentralized and mobile edge storage solutions.

Criteria	Handheld	R-Drive	MDFS	DistressNet-NG	FogStorage
Fault tolerance	Moderate (replication)	Very high (erasure coding)	High (erasure coding)	Very high (hybrid)	Moderate (replication + local caching)
Security	None	AES-256 + SSS	AES	AES (inherited)	AES-based encryption
Energy mgmt	Battery Manager	Partial	None	Partial	Not optimized (fixed nodes)
Scalability	~10 devices	Medium	Small	Medium	Limited (indoor/fixed environment)
Connectivity	Requires AP	Opportunistic (RSocket)	Ad hoc (MANET)	Semi-fixed nodes	Indoor/fog network nodes
Key limitation	AP-dependent	Semi-fixed nodes	No DHT, no energy	Tactical niche only	Fixed indoor environment

3.5 Enabling Technologies for Next-Generation Mobile Storage

Beyond the prototypes reviewed above, two foundational technologies have emerged that could address the identified limitations: conflict-free data consistency and predictive fault tolerance.

3.5.1 Conflict-Free Replicated Data Types (CRDTs)

CRDTs [41] guarantee *eventual consistency* of replicas modified independently, without central coordination or consensus. They come in two families — state-based (CvRDTs) and operation-based (CmRDTs) — while delta-state CRDTs [42] transmit only incremental changes, which is critical for bandwidth-constrained mobile links (BLE: 1–3 Mbps; Wi-Fi Direct: ~250 Mbps). Kleppmann [43] characterizes CRDTs as a shift from “coordinate then act” to “act then converge,” making them well suited to offline-first mobile P2P systems; in particular, LWW-Element-Sets with Tombstones solve the *data resurrection problem*, where a file deleted during a partition would otherwise reappear after healing.

Gap: Despite their strong theoretical foundations, CRDTs have never been implemented in a mobile P2P *storage* system with proper deletion handling via Tombstones.

3.5.2 AI-Driven Predictive Resilience

Alahmadi et al. [44] propose a proactive fault-tolerance framework that, unlike reactive mechanisms (replication, redundancy, failover), aims to predict failures before they occur. Using

supervised models (Random Forest, Gradient Boosting) trained on historical metrics such as CPU, memory, and disk usage, they detect potential failures with $\sim 70\%$ accuracy, enabling preventive actions such as resource reallocation or task migration.

Gap: This approach targets cloud-based infrastructures with abundant computational resources, not the dynamic, resource-constrained setting of mobile storage. No existing mobile storage solution leverages lightweight on-device ML to predict node departures and proactively redistribute data before disconnection.

3.6 Identification of Research Gaps

Table 3.2 formalizes the five specific research gaps identified across all reviewed systems.

Tableau 3.2: Identified research gaps in distributed mobile storage systems.

Gap	Description	Affected Systems
Persistent storage on smartphones	No system guarantees persistent data availability using erasure coding on smartphones as <i>exclusive</i> storage nodes, without fixed infrastructure.	All
Scalable DHT on mobile ad hoc networks	No system has validated DHT $O(\log N)$ routing at scale ($>1,000$ nodes) on a purely mobile ad hoc network with high churn.	Kademlia, Chord, IPFS
Post-partition consistency	No mobile storage system integrates CRDTs with Tombstones to prevent data resurrection after partition healing.	R-Drive, MDFS, DistressNet-NG
Predictive resilience via on-device AI	No system uses embedded ML on smartphones to predict node churn and proactively redistribute data before failure.	All
Large-scale validation	No mobile storage prototype has been validated via simulation ($>10,000$ nodes) with realistic churn injection.	R-Drive, MDFS, DistressNet-NG

The analysis throughout this chapter reveals three families of systems, each addressing only a subset of the requirements for resilient distributed storage on mobile devices at scale:

- **Family A — Scalable but non-mobile:** HDFS, Ceph, Kademlia, Chord, and IPFS demonstrate proven scalability but assume stable, continuously powered nodes, and none operates autonomously on smartphones in ad hoc networks.
- **Family B — Mobile but non-scalable:** MDFS, R-Drive, DistressNet-NG, and the Lisbon handheld system demonstrate feasibility on Android but have not been evaluated beyond a few dozen nodes, mostly relying on semi-fixed edge nodes.

-
- **Family C — Intelligent but non-decentralized:** CRDTs [41, 43] and AI/ML-driven fault prediction [44] offer advanced resilience but have only been applied in centralized or semi-managed environments. No system combines on-device AI with CRDTs on purely mobile P2P nodes.

These gaps define an unexplored intersection between Big Data scalability, mobile ad hoc constraints, and modern AI-driven resilience mechanisms.

3.7 Conclusion

This state-of-the-art review shows that while large-scale distributed storage has reached maturity in data center environments, its adaptation to mobile ad hoc networks remains a largely open problem. Three fundamental tensions emerge: **scalability vs. mobility**, since Big Data systems (HDFS, Ceph) are proven at scale but assume fixed nodes whereas mobile prototypes (R-Drive, MDFS, DistressNet-NG) remain limited to a few dozen devices; **consistency vs. decentralization**, since CRDTs provide coordination-free consistency but have never been implemented in a mobile P2P storage system with proper deletion semantics; and **reactive vs. predictive resilience**, since all existing mobile systems react to failures rather than anticipating node departures through on-device learning.

To our knowledge, no existing system simultaneously provides: (1) Big Data scalability via DHT $O(\log N)$ routing validated at large scale, (2) operation exclusively on mobile terminals via ad hoc protocols, (3) persistent storage via adaptive erasure coding, (4) post-partition consistency via CRDTs with Tombstones, and (5) predictive resilience through embedded AI inference — all without any dependency on Internet connectivity or fixed infrastructure. The present work aims to bridge this gap; the following chapter presents the proposed architecture designed to address these shortcomings.

4 Proposed Solution Design

4.1 Introduction

This chapter presents the design of the proposed distributed mobile storage system. It is organized into two main parts. The first part presents the proposed solution: it defines the system’s objectives and the constraints imposed by the mobile environment, describes the global network architecture, and details the core algorithms and design decisions that govern the system’s runtime behavior, organized around five functional domains. The second part formalizes the design through UML models: use case diagrams that capture the functional scope of the system, a class diagram that describes its static structure, sequence diagrams that detail the critical operational scenarios, and state machine diagrams that specify the autonomous behavior of participating nodes.

Part 1 : Proposed Solution

4.2 System Overview

The proposed system is a distributed storage architecture that transforms a federation of ordinary smartphones into a global, resilient, and privacy-preserving storage network. It represents a decentralized alternative to conventional cloud storage infrastructures, where data sovereignty and availability are guaranteed by the collective participation of mobile devices rather than by a trusted central authority. Unlike traditional distributed systems designed for stable server environments, this system operates in a highly dynamic mobile context, where frequent node disconnections, a phenomenon referred to as *churn*¹ —are treated as an inherent operating condition rather than an exception.

Mobile Environment Constraints However, deploying a distributed storage system on smartphones introduces fundamental challenges inherent to mobile environments. First, **Node Volatility** represents a major constraint, as mobile devices are inherently unstable and may disconnect at any time due to mobility, energy depletion, or connectivity disruptions. Second, **Resource Limitation** significantly impacts system design, since smartphones are constrained in terms of processing power, storage capacity, and battery energy, thereby limiting the cost of distributed operations. Third, **Device Heterogeneity** introduces additional complexity, as participating nodes exhibit significant variations in hardware capabilities, network quality, and overall performance. Furthermore, **Platform Restrictions**, imposed by mobile operating systems such as

¹Churn denotes the continuous process of nodes joining and leaving a distributed network, a particularly prevalent condition in mobile environments due to battery constraints, mobility, and intermittent connectivity.

Android, limit background execution and long-running services, making continuous coordination more challenging. Consequently, these combined constraints define a highly dynamic and unpredictable environment, which requires the system to be inherently **Adaptive**, **Lightweight**, and **Fault Tolerant**.

System Objectives In response to these constraints, the system is designed to achieve the following key objectives:

- **Resilience:** ensuring continuous data availability despite frequent node departures and failures
- **Adaptability:** dynamically adjusting redundancy and coordination behavior according to evolving network conditions
- **Security:** guaranteeing confidentiality and integrity of stored data in an untrusted distributed environment
- **Transparency:** abstracting all distributed complexities from the end user, providing a seamless storage experience
- **Scalability:** supporting clusters of varying sizes and heterogeneous devices without performance degradation

Design Principles To achieve these objectives, the architecture relies on a set of fundamental design principles.

Zero-Knowledge Storage: ensures that files are encrypted before leaving the owner's device. No storage node can access the content of the fragments it hosts, even if compromised, confidentiality is guaranteed by cryptographic construction, not by trust.

Adaptive Erasure Coding : replaces traditional replication as the core redundancy mechanism. Data is divided into K fragments and encoded with N additional parity fragments, allowing full reconstruction from any K out of $K + N$ fragments. The redundancy level is dynamically adjusted to reflect the current reliability of the cluster, significantly reducing storage overhead while maintaining strong fault tolerance.

Reliability-Aware Management: continuously monitors node conditions to guide data placement decisions and proactively anticipate node departures before they result in data loss.

4.3 Global Architecture

4.3.1 Node Types

The system mainly relies on two types of physical devices playing complementary roles within the distributed architecture:

- **Cluster Head** : the Cluster Head is a mobile device dynamically elected among the devices of a cluster. Physically, it is an ordinary smartphone temporarily acting as the local coordinator of the cluster. Its role is to orchestrate the activities of the group, manage active members, coordinate communications, and supervise data distribution. In case of failure, a new election is automatically triggered to designate a new coordinator.
- **Node** : a node is a mobile device participating in the distributed storage network. It contributes by providing a portion of its local storage capacity. Its role is to store encrypted data fragments, participate in network exchanges, discover nodes, and provide or retrieve data blocks whenever required by the system.

4.3.2 Network Topology

This solution adopts a hybrid architecture that combines a minimal centralized entry point with a fully decentralized data governance model. This design reflects a deliberate trade-off : preserving the simplicity of initial network discovery while ensuring that no single authority controls data storage, retrieval, or coordination. As illustrated in Figure 4.1.

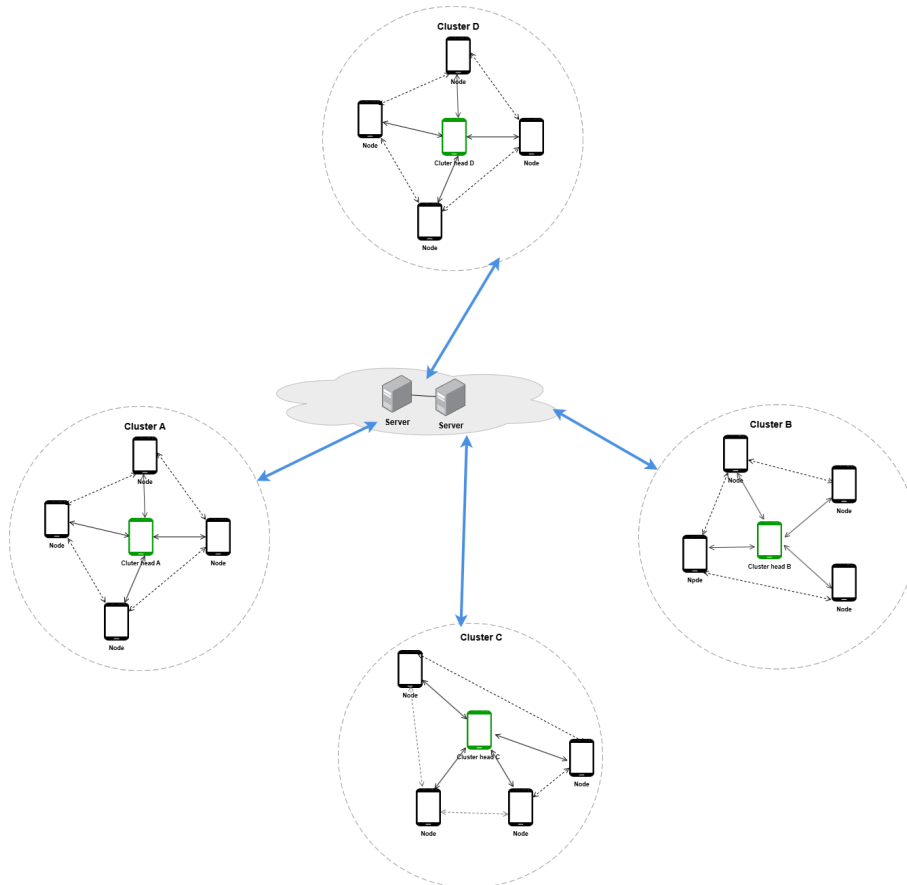


Figure 4.1: Network topology of the system.

Relay Server The Relay Server is the only fixed infrastructure component of the system. It acts as the central coordination point of the network and never stores any user data. Before any exchange, each node must authenticate itself to the Relay Server in order to be admitted to the network. It communicates exclusively with the cluster head of the network, with whom it exchanges cluster registration messages, the list of known nodes, relayed messages between clusters, and coordination signals.

Cluster head The Cluster head is a mobile node dynamically elected within each cluster to act as its local coordinator. It represents the cluster to the outside world and manages its internal organization. It communicates with three actors: the Relay Server, with whom it exchanges registration, node discovery, and relayed messages; the Cluster head of other clusters, with whom it exchanges inter-cluster data transfers and coordination messages; and the ordinary nodes of its own cluster, with whom it exchanges data distribution, membership management, and availability monitoring.

Ordinary Node An ordinary node is a standard device that contributes its available storage to the system by hosting encrypted file fragments. It communicates with two actors: its cluster head, with whom it exchanges data and coordination signals; and the other nodes of the same cluster, with whom it exchanges file fragments.

4.3.3 Layered Software Architecture

The system follows a **Clean Architecture** organized into four layers with strictly unidirectional dependencies. Each layer has a single responsibility and depends only on the layer below it, ensuring that the distributed coordination logic remains independent of the user interface and the technical infrastructure, As illustrated in Figure 4.2.

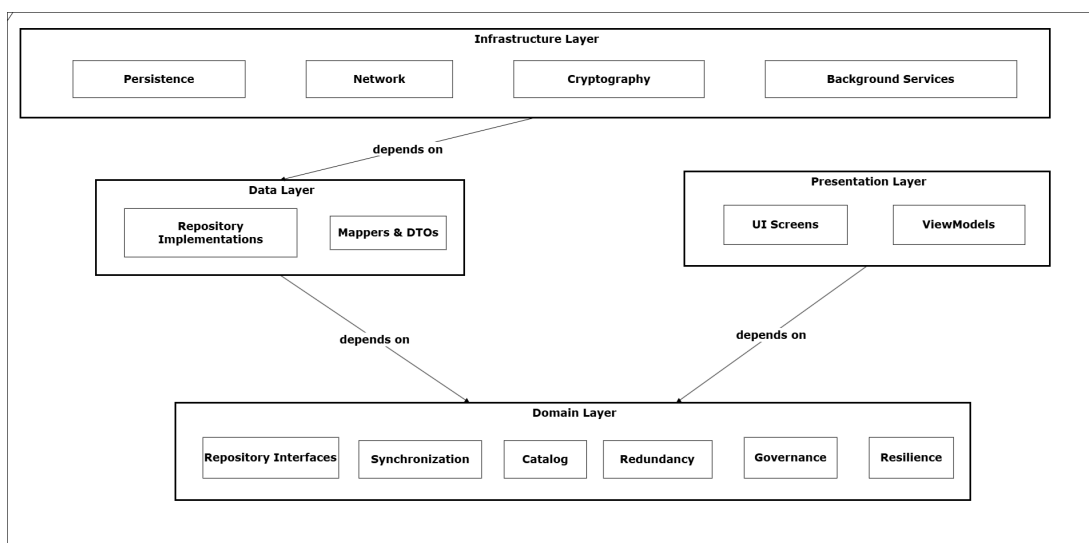


Figure 4.2: Layered Software Architecture

-
- **Presentation Layer** : manages the user interface and translates user actions into requests addressed to the domain layer. It contains no distributed algorithm.
 - **Domain Layer**: constitutes the conceptual core of the system. It defines the business models and the distributed protocols: peer discovery, leader election, erasure coding, fragment placement, gossip synchronization, and the security pipeline.
 - **Data Layer**: provides the concrete implementations of the domain contracts. It encapsulates local persistence and all network communication transports used by the system.
 - **Infrastructure Layer**: centralizes the low-level technical foundations shared across all layers: cryptographic primitives, hardware-backed identity management, the native erasure coding engine, and the network state observer.

The composition of these layers is orchestrated through a dependency injection mechanism that wires concrete implementations to abstract contracts at application startup, enforcing architectural boundaries throughout the life cycle of the application.

4.4 Core Algorithms & Design Decisions

This section presents the core algorithms and design decisions that govern the runtime behavior of the system. Rather than describing these algorithms in isolation, they are organized around five functional domains that reflect the fundamental questions any distributed storage system must answer: *who composes the network?* (Membership), *what is stored and how?* (Storage), *where is the data and how to find it?* (Localization and Routing), *how does the system survive failures?* (Resilience), and *who has the right to access the data?* (Security). Each domain groups the algorithms that collectively address one of these concerns, making the design intent explicit before the technical details are presented.

Membership : Who composes the network?

4.4.1 Node Discovery & Cluster Formation

Before any storage or coordination activity can take place, a node must locate existing participants and integrate itself into a cluster. This bootstrapping process is entirely self-managed: no prior knowledge of the network topology is assumed, and no manual configuration is required from the user.

4.4.1.1 Discovery Mechanism

Node discovery is handled exclusively through the relay server, which serves as the single fixed point of contact in the network. This design choice is deliberate: in a mobile environment where devices change addresses frequently and may reside on entirely different networks, a shared

and always-reachable directory is the only reliable means of introducing nodes that have never previously met. The relay maintains a continuously refreshed registry of active cluster head across all known clusters. When a node starts or loses its cluster, it queries this registry to obtain a list of reachable coordinators. The relay enforces a short expiration window on its entries, ensuring that only currently active cluster head appear in the response. A node never sees itself in the returned list.

4.4.1.2 Cluster Selection

Once candidate clusters are identified, the node applies a *Cluster Affinity* policy to determine which cluster to contact. Historical affinity takes first priority: a node always attempts to rejoin its previous cluster to preserve data locality and avoid unnecessary redistribution.

If that cluster is unavailable, the node selects the least populated candidate to balance load evenly across the network. When several clusters are equally sized, the choice is resolved by a deterministic lexicographic comparison of Cluster head identifiers, ensuring that all nodes observing the same candidates independently converge on the same selection without coordination. Clusters that have reached their capacity limit are excluded before any of these rules are applied.

4.4.1.3 Admission Protocol

The candidate transmits a signed admission request to the selected Cluster head, carrying its cryptographic identity, available storage capacity, and a timestamped payload. The cluster head evaluates the request through four sequential checks, each of which independently may cause a refusal:

- **Coordinator status** : the cluster head confirms it is currently in an active coordination state. A node undergoing an election or transition cannot safely admit new members.
- **Signature validity** — the cryptographic signature of the request is verified against the public key declared by the candidate, ensuring the request genuinely originates from the claiming node.
- **Timestamp freshness** — the request timestamp must fall within a narrow freshness window, neutralizing replay attacks in which an adversary resubmits a previously captured valid request.
- **Cluster capacity** — the cluster must not have reached its maximum size. The verification and the subsequent membership update are performed atomically to safely handle concurrent admission requests.

When all checks pass, the candidate is admitted, existing members are notified, and the new node receives a signed membership snapshot enabling it to participate immediately.

Refusals carry a semantic distinction: a *soft refusal* caused by a saturated cluster or an unavailable coordinator, includes alternative cluster head for the candidate to contact, while a *hard refusal* caused by a cryptographic failure contains no alternatives, as the problem originates from the candidate itself.

4.4.1.4 Failure Handling and Node Life cycle

When all admission attempts are exhausted, the node enters an *isolation state* and monitors the relay passively. If a reachable cluster head appears, the process resumes immediately. Otherwise, the node triggers a *cluster genesis procedure*: it elects itself as the initial cluster-head, generates a **new cluster identifier**, and registers with the relay to become discoverable by other participants.

The complete lifecycle of a node is formalized as a finite state machine of six states: *Undiscovered*, *Joining*, *Member*, *Rejoining*, *Isolated*, and *cluster-head*. This model ensures that a node always occupies a well-defined role and that no invalid transition is possible. The principal transitions are summarized in Table 4.1.

Transition	Trigger
Undiscovered → Joining	cluster-head detected via relay
Joining → Member	Admission accepted
Joining → Isolated	All attempts exhausted
Isolated → cluster-head	Solo election won
Member → Rejoining	cluster-head unreachable
Rejoining → cluster-head	Bully election won
Rejoining → Member	COORDINATOR message received
cluster-head → Undiscovered	Abdication or election lost

Tableau 4.1: Principal transitions of the node lifecycle finite state machine.

4.4.2 Cluster head Election

Within each cluster, exactly one node must assume the coordinator role the cluster-head. Since any mobile device can disconnect or degrade at any time, the system requires a mechanism to designate this coordinator dynamically and to replace it automatically when necessary. The system adopts the **Bully Algorithm** [?], a classical distributed election protocol adapted to the mobile context by replacing the static process identifier with a dynamic **Reliability Score**.

Reliability Score Each node computes a reliability score that reflects its current capacity to sustain the coordinator role. This score is defined as a weighted linear combination of three normalized factors:

$$S = \alpha \times \text{Battery} + \beta \times \text{Uptime} + \gamma \times \text{Network}, \quad \alpha + \beta + \gamma = 1, \quad S \in [0.0, 1.0] \quad (4.1)$$

The variables of Equation 4.1 are defined as follows:

- S : the reliability score of the node, bounded in $[0.0, 1.0]$. It serves as a single election criterion: the node with the highest S wins the election.
- α, β, γ : configurable weights associated with each factor, subject to the constraint $\alpha + \beta + \gamma = 1$, which ensures that S remains normalized regardless of the individual factor values.
- **Battery** : the current battery level of the node, expressed as a value in $[0, 1]$. The battery carries the highest default weight because the role of the cluster-head demands sustained processing and network activity, which translates directly into energy consumption on a mobile device.
- **Uptime**: the continuous availability duration of the node, normalized to $[0, 1]$ by dividing the elapsed runtime by a reference stability horizon that represents a fully stable node. The result is fixed at 1.0 for nodes that exceed this horizon. This factor rewards nodes that have remained connected without interruption and penalizes devices that frequently connect and disconnect.
- **Network** — the quality of the node's network connection, normalized to $[0, 1]$. It reflects the coordinator's ability to reliably exchange messages with all cluster members. The normalization method and the specific values assigned to each connectivity type are detailed in Chapter 5.

A critical safeguard, referred to as the **Guillotine rule**, applies unconditionally: when a node's battery falls below a critical threshold, its score is immediately forced to zero regardless of the other factors. This prevents a device on the verge of shutdown from winning or retaining the coordinator role, protecting the cluster from a predictable and imminent failure of leadership.

In the event of equal scores, the tie is broken deterministically by lexicographic comparison of node identifiers, guaranteeing a unique winner without requiring additional communication.

Election Protocol An election is initiated in three situations:

- The cluster-head has been absent beyond the monitoring window — no coordination signal has been received for a sustained period.
- A node starts and finds no active coordinator in the cluster — it either founds a new cluster alone or triggers an election among available nodes.

- The current cluster-head detects that its own reliability score has fallen below a critical threshold — it voluntarily initiates its replacement rather than waiting for a failure to be detected externally.

The election proceeds in four steps. First, the initiating node broadcasts an **ELECTION** message containing its current score to all known nodes. Second, each receiving node compares the sender’s score with its own: a stronger node replies with an **ALIVE** message and simultaneously launches its own candidacy; a weaker node remains silent. Third, the initiator waits for any **ALIVE** response within a fixed timeout window. Fourth, if no stronger competitor responds, the node declares victory and broadcasts a signed **COORDINATOR** message to the entire cluster, assuming the cluster-head role immediately.

Mandate, Rotation, and Abdication Once elected, the cluster-head holds its role for a **bounded mandate**. At the end of this period, it voluntarily steps down even if it remains fully operational. This forced rotation is a deliberate design decision motivated by fairness in resource consumption: on a smartphone, acting as cluster-head incurs a sustained overhead in battery consumption and heat production. If the same device held this role indefinitely, its owner would alone bear the coordination cost for the entire network. Periodic rotation distributes this cost equitably across the most capable nodes over time.

The step-down procedure consists of broadcasting an **ABDICATION** message to all cluster members, which triggers an immediate and orderly new election. After abdicating, the node enters a **cooldown period** during which it is ineligible to re-candidate. This prevents the same node from immediately reclaiming the role it just vacated, reinforcing the rotation effect.

The cluster-head may also abdicate before the end of its mandate in two cases: when its reliability score falls below the critical threshold defined by the Guillotine rule, or when it shuts down gracefully. In both cases, the abdication message allows the cluster to react immediately rather than waiting for the failure detection timeout to expire.

Table 4.2 summarizes the key parameters of the election protocol.

Parameter	Design Decision
Election criterion	Highest reliability score S
Score formula	$\alpha \cdot \text{Battery} + \beta \cdot \text{Uptime} + \gamma \cdot \text{Network}$
Tie-breaking rule	Lexicographic comparison of node identifier
Guillotine rule	Score forced to 0 below critical battery threshold
Election triggers	SP absence, node isolation, score below threshold
Mandate	Bounded and enforced
Rotation	Forced at end of mandate
Cooldown after abdication	Node ineligible to re-candidate immediately

Tableau 4.2: Summary of the leader election protocol design decisions.

4.4.3 Failure Detection

In a distributed mobile system, failure cannot be observed directly ; it can only be inferred from the sustained absence of expected communication. The system applies **timeout-based failure detection** through a bidirectional heartbeat protocol that operates in both directions between members and their cluster-head.

Heartbeat Mechanism Each member periodically transmits a signed heartbeat to its cluster-head. This message serves a dual purpose: it signals the continued presence of the node, and it carries useful metadata about the node's current state, such as available storage capacity and network address allowing the cluster-head to maintain an up-to-date view of its cluster without requiring separate synchronization messages.

Symmetrically, the cluster-head also transmits periodic signals to its members, confirming that the coordinator remains operational. Members track the timestamp of the last signal received from their coordinator and use it as the basis for liveness evaluation.

Member Liveness The cluster-head maintains a registry of last heartbeat timestamps for each cluster member and periodically evaluates whether each entry remains within the acceptable silence window. A member that exceeds the eviction threshold is removed from the cluster registry. The eviction check is performed atomically to prevent false evictions caused by heartbeats that arrive during the evaluation tick.

When a member is evicted, the cluster-head assesses whether the departure creates an under-replication situation for any hosted fragment and, if so, triggers the self-healing mechanism described in Section 4.4.5.

Cluster-head Liveness A member concludes that its cluster-head has failed only when two conditions are simultaneously satisfied: the cluster-head has been silent beyond the suspicion threshold, *and* the member has not experienced local connectivity errors during the same period. This double condition is a deliberate design decision , it allows the system to distinguish a genuine cluster-head failure from a transient local network disruption on the member's own side. Without this distinction, a member with a momentary connectivity issue would incorrectly trigger a cluster-wide election, destabilizing the entire cluster for a problem that only affects one node.

When both conditions are met, the member transitions to the *Rejoining* state and initiates a Bully election to designate a new coordinator.

Voluntary Departure A cluster-head that shuts down gracefully, due to low battery, score below the critical threshold, or application closure, broadcasts an ABDICATION message before disconnecting. All members react immediately upon reception, without waiting for the suspicion

timeout to expire. This transforms a planned departure into an orderly leadership transition and minimizes the period during which the cluster operates without a coordinator.

Relay Failure When the relay server becomes unreachable, nodes lose the ability to exchange signaling messages across network boundaries. The system responds with automatic reconnection, resuming normal operation as soon as connectivity is restored.

Summary Table 4.3 summarizes the four failure scenarios handled by the system, the detection mechanism applied in each case, and the resulting reaction.

Failure	Detection	Reaction
Member node	Heartbeat absent beyond eviction threshold	Silent eviction; self-healing activated if needed
cluster-head crash	SP silent beyond suspicion threshold <i>and</i> no local connectivity error	Member initiates Bully election
cluster-head voluntary	ABDICATION message received	Immediate election without timeout
Relay server	WebSocket disconnection	Automatic reconnection

Tableau 4.3: Failure scenarios, detection mechanisms, and reactions.

4.4.4 Erasure Coding

The system uses **Reed-Solomon erasure coding** over the Galois field of 256 elements $GF(256)$ to provide data redundancy across the cluster. This approach replaces traditional block replication, offering equivalent fault tolerance at a significantly lower storage cost.

Replication vs Erasure Coding Table 4.4 compares the three redundancy strategies that could be applied in this context.

Approach	Redundancy	Storage Cost	Tolerated Failures
No redundancy	None	$1 \times$ file size	0
Triple replication	3 full copies	$3 \times$ file size	2
Erasure Coding (K, N)	$K + N$ fragments	$\frac{K+N}{K} \times$ file size	N

Tableau 4.4: Comparison of redundancy strategies.

With erasure coding, any K fragments out of the $K + N$ distributed are sufficient to reconstruct the original file. The remaining N fragments are parity fragments that provide redundancy. Both

K and N are configurable parameters, subject to the constraint imposed by the mathematical structure of GF(256):

$$K + N \leq 255 \quad (4.2)$$

This bound arises from the fact that GF(256) contains exactly 256 elements, limiting the maximum number of distinct fragments that can be produced by the encoding matrix.

Adaptive Redundancy Rather than applying a fixed redundancy configuration, the system dynamically adjusts N based on the observed reliability of the current cluster. This adaptation ensures that a stable cluster does not waste storage on unnecessary redundancy, while an unstable cluster receives stronger protection. Table 4.5 presents the three redundancy profiles.

Cluster Reliability Score	Parity Blocks (N)	Tolerated Failures
≥ 0.8 (stable)	1	Any 1 loss out of $K + 1$
≥ 0.5 (normal)	2	Any 2 losses out of $K + 2$
< 0.5 (unstable)	4	Any 4 losses out of $K + 4$

Tableau 4.5: Adaptive redundancy profiles as a function of cluster reliability.

Encoding and Decoding The encoding process divides the source file into K equal data blocks and applies Reed-Solomon matrix algebra over GF(256) to produce N additional parity blocks. The result is an indexed array of $K + N$ fragments: indices $[0, K)$ carry the original data, and indices $[K, K + N)$ carry the parity.

The decoding process requires any K fragments from the $K + N$ available, regardless of whether they are data or parity fragments. The decoder identifies the surviving fragments by their indices, inverts the corresponding submatrix of the encoding matrix, and reconstructs the K original data blocks. This property that any K out of $K + N$ fragments suffice, is the fundamental guarantee of Reed-Solomon codes and the basis for the system’s fault tolerance model.

4.4.5 Fragment Placement and Self-Healing

Once a file has been encoded into $K + N$ fragments, the system must decide which nodes will host each fragment and how to restore the required redundancy level when a hosting node becomes unavailable.

Fragment Placement The placement algorithm selects the $K + N$ recipient nodes through a two-step strategy. First, a **capacity filter** retains only nodes with sufficient free storage to host a fragment, including a safety margin for metadata overhead. Second, the eligible nodes are ranked by decreasing reliability score, and the top $K + N$ nodes are selected, each receiving a distinct fragment.

Two principles guide this strategy. **Reliability-first placement** concentrates the distribution on the most stable nodes in the cluster, maximizing the probability that at least K fragments remain simultaneously available over time. **Fragment diversity** ensures that no two fragments are assigned to the same node, so that the loss of a single node can never cause the loss of more than one fragment.

Under-Replication Detection The system detects under-replication through two distinct paths, each corresponding to a different type of node departure.

The first path handles **silent departures**. When a node exceeds the heartbeat silence threshold, it is marked inactive in the cluster registry. A periodic scan then consults the distributed catalog to identify which fragments were hosted by the inactive node, counts the remaining active hosts for each affected fragment, and flags any fragment whose number of active hosts falls below the required threshold as under-replicated.

The second path handles **graceful departures**. When a node shuts down intentionally, it broadcasts a signed departure notice containing the list of fragments it currently hosts. Upon reception, the cluster-head immediately constructs a migration plan without waiting for the heartbeat timeout to expire. This path significantly reduces the repair latency for planned departures compared to the silent departure path.

Self-Healing When an under-replicated fragment is detected through either path, the cluster-head constructs a **migration plan** specifying, for each affected fragment, the source node and the destination node selected to receive the replica. The destination is chosen by applying the same placement strategy described above — reliability-first, with diversity enforcement — among the nodes that do not already host a copy of the fragment.

The source node executes the migration directives in parallel, bounded by a concurrency limit to avoid overwhelming the network with simultaneous transfers.

Circuit Breaker : Anti-Churn Protection When multiple nodes leave the cluster in rapid succession, a situation known as a *churn storm*, the naive response would be to emit a separate migration plan for each lost fragment simultaneously. This approach risks saturating the network with repair traffic precisely when the cluster is already under stress, potentially worsening the situation it is trying to correct.

The system addresses this through a **Circuit Breaker** mechanism. When the rate of detected failures exceeds a defined threshold within a short observation window, the system temporarily suspends the emission of new migration plans and enters a stabilization period. During this period, it continues to monitor the cluster and accumulate the list of under-replicated fragments, but defers all repair actions until the churn rate returns to a normal level. Once the cluster stabilizes, a consolidated repair plan is emitted, addressing all accumulated under-replications in a controlled and prioritized manner.

This mechanism reflects a deliberate architectural trade-off: accepting a temporary increase in vulnerability in exchange for avoiding a repair storm that could cause cascading failures across an already degraded cluster.

4.4.6 Distributed Catalog

Once fragments have been distributed across the cluster, the system must be able to answer a fundamental question at any time: *where is a given fragment, and on which node can it be retrieved?* This is the responsibility of the **distributed catalog**, which maintains a consistent mapping between fragment identifiers and their current hosting nodes across all members of the cluster.

Distributed Hash Table Design The catalog is implemented as a **Distributed Hash Table (DHT)** in which each entry maps a fragment identifier, computed by a hash function applied to the fragment, to the node that currently hosts it, identified by its node identifier and network address. A timestamp is associated with each entry to detect stale records and resolve update ordering.

The design anticipates two distinct access patterns, each corresponding to a critical operation of the system:

- **Fragment lookup:** given a fragment identifier, find the node that hosts it. This path is triggered at every file retrieval operation and must be resolved efficiently.
- **Node eviction cleanup:** given a node identifier, find all fragments it hosts. This path is triggered whenever a node is evicted from the cluster, and must allow the self-healing mechanism to immediately identify which fragments have become under-replicated.

Supporting both access patterns efficiently requires the catalog to be indexed along both dimensions simultaneously, a design decision that directly conditions the responsiveness of both the retrieval pipeline and the repair pipeline.

Conflict Resolution Since the catalog is replicated across multiple nodes through gossip synchronization, concurrent updates may produce conflicting versions of the same entry. The system resolves these conflicts through a **deterministic merge function** based on a logical clock: the entry with the higher logical timestamp is retained. When timestamps are equal, the conflict is resolved by a secondary deterministic rule that guarantees a unique outcome regardless of the order in which updates are received. The details of this merge function are described in Section 4.4.7, where the full structure of a catalog entry is presented.

This conflict resolution strategy belongs to the family of **Last-Write-Wins Conflict-free Replicated Data Types (CRDTs)**, which guarantee eventual consistency: all nodes that receive the same set of updates will converge to the same catalog state, regardless of the order in which those updates arrive.

4.4.7 Catalog Data Structure

The Catalog is the central data structure of the distributed catalog. Each entry describes a single stored file from the perspective of the storage system: where its fragments are located, how they were encoded, and how their encryption key can be recovered by the owner. Table 4.6 presents the fields of this structure and their respective roles.

Field	Role
fileHash	Unique identifier of the file, derived from a cryptographic digest of its content. Serves as the catalog key.
ownerPubKeyHash	Cryptographic digest of the owner's public key. Allows ownership to be verified without revealing the owner's identity.
versionClock	Logical clock (Lamport clock) used by the conflict resolution mechanism during gossip synchronization. When two versions of the same entry are received, the one with the higher clock value is retained.
fragmentLocations	Routing table mapping each fragment index to the identifier and network address of its current hosting node. Updated whenever a fragment is migrated to a new host.
wrappedMasterKey	The file's encryption master key, encapsulated using an asymmetric key transport scheme. Only the holder of the owner's private key can unwrap it.
originalFileSize	The exact byte length of the original file, required to remove the padding introduced during erasure encoding and reconstruct the file at its exact original size.
k, n	The erasure coding parameters used at encoding time, preserved to guide future decoding operations.

Tableau 4.6: Fields of the CatalogEntry structure and their roles.

The `version Clock` field deserves particular attention, as it is the mechanism through which the catalog achieves eventual consistency. Lamport clocks are a classical solution to the problem of ordering events in distributed systems where no global clock is available. Each update to a catalog entry increments the clock, and the conflict resolution rule — retain the entry with the higher clock value — guarantees that all nodes converge to the same catalog state regardless of the order in which updates are received. When two entries carry equal clock values, a secondary deterministic rule based on the fragment identifiers breaks the tie, ensuring that the merge outcome is unique and reproducible on every node.

4.4.8 Gossip Protocol and Bloom Filter Delta-Sync

The distributed catalog and the distributed hash table are replicated across all nodes of the cluster. Without a synchronization mechanism, a node that joins the cluster, or one that was

temporarily disconnected, would hold an outdated view of the catalog — unaware of fragments that were added, migrated, or removed during its absence. Propagating every update to every node via direct broadcast would generate a volume of traffic proportional to the square of the cluster size, which is incompatible with the bandwidth constraints of mobile devices.

The system addresses this through a **gossip protocol**, an epidemic dissemination mechanism in which each node periodically shares its local state with a small random subset of its nodes. Over successive cycles, updates propagate progressively across the cluster until every node has received them, without any node needing a global view of the network or a direct connection to every other node.

Bloom Filter Delta-Sync A naive gossip implementation would transmit the full catalog at every cycle, which remains costly for large catalogs. The system optimizes this through **delta synchronization** using Bloom filters.

A Bloom filter is a compact probabilistic data structure that answers the question: *is this element in the set?* It represents a set of identifiers as a fixed-size bit array, using multiple hash functions to map each identifier to a set of bit positions. Membership queries return either *definitely not present* or *probably present*, with a controllable false-positive rate. The key property for this use case is that the size of the filter is constant — it does not grow with the number of elements it represents.

Each gossip cycle proceeds in three phases. First, the node encodes the identifiers of all its local DHT entries into a Bloom filter and transmits it to a small random subset of active nodes. Second, each receiving node compares the filter against its own DHT: entries that are absent from the filter are identified as candidates for propagation. Third, the receiver sends back only the missing entries as a delta response, avoiding the retransmission of data the initiator already holds.

This approach reduces the synchronization message size from $O(|\text{DHT}|)$ to a constant, regardless of how many entries the catalog contains. Only genuinely missing entries are exchanged, making the per-cycle bandwidth cost independent of the total catalog size.

Security To prevent catalog poisoning, only authenticated nodes can inject entries into the DHT. A node that receives a gossip message from an unknown or unverified source discards it without processing, limiting the attack surface to nodes that have been formally admitted to the cluster.

Convergence The gossip protocol provides **eventual consistency**: every live node will eventually receive every update, provided the cluster remains connected over time. The combination of random node selection and a bounded fan-out per cycle, where each node contacts only a small fixed number of nodes rather than broadcasting to all, ensures that updates propagate in $O(\log N)$ cycles, where N is the number of nodes in the cluster. This logarithmic convergence

means that even as the cluster grows, the time required for a catalog update to reach all nodes grows only slowly, keeping the synchronization overhead predictable and manageable.

4.4.9 Security Pipeline

The security pipeline is designed around a single governing principle: **zero-knowledge storage**. No node that participates in the system, including the cluster-head and the relay server, can access the content of the data it handles. This property is not enforced by policy or trust, but by cryptographic construction: the data is encrypted before it leaves the owner's device, and only the owner holds the key required to decrypt it.

Node Identity Each node possesses a cryptographic identity generated inside the secure hardware of the device. This key is non-exportable and non-duplicable by construction: it is created within the hardware security boundary and never leaves it, even in plaintext form. An attacker who gains control of the operating system cannot extract this key, and cannot therefore create multiple valid identities on virtual machines or cloned devices. This hardware anchoring is the foundation of the system's resistance to Sybil attacks, in which an adversary would attempt to flood the network with fraudulent identities to subvert the election or the fragment placement mechanisms.

All protocol messages are signed with this hardware-backed key, binding every communication to a verifiable, non-forgable identity.

Fragment Encryption Each file is encrypted using a **two-level key hierarchy**. At the first level, a unique master key is generated for the file. At the second level, a distinct derived key is used to encrypt each individual fragment independently.

This hierarchy is a deliberate design decision that addresses two distinct security requirements. First, it enables **controlled access sharing**: a file owner can grant another user access to a file by sharing only the master key, without exposing their own private key. Second, it enforces **fragment-level isolation**: the compromise of a single fragment — and its associated derived key — does not expose the master key or any other fragment of the file. Each fragment is cryptographically independent from all others.

Key Transport The master key must be stored somewhere accessible to the owner at retrieval time, yet unreadable to any storage node. The system achieves this by **encapsulating the master key using an asymmetric scheme** based on the owner's public key. The encapsulated key is stored in the `wrappedMasterKey` field of the `CatalogEntry` and propagated across the cluster through gossip synchronization. Any node can hold this field as an opaque blob — it cannot derive any information from it without the owner's private key, which never leaves the owner's hardware security boundary.

Message Authentication and Replay Protection Every network message in the system — discovery announcements, election messages, heartbeats, admission requests, and migration plans — is signed with the sender’s private key. The signature provides two guarantees: **authenticity**, confirming that the message originates from the claimed sender, and **integrity**, ensuring that the message has not been altered in transit. Any message whose signature verification fails is silently discarded.

A timestamp is attached to every message and verified upon reception. Any message whose timestamp falls outside a defined freshness window is rejected as potentially replayed. Combined with the signature — which binds the timestamp to a specific sender and a specific payload — this mechanism prevents an adversary from capturing a legitimate message and resubmitting it at a later time to produce an unintended effect.

4.5 System Requirements

4.5.1 Actors

The system involves two distinct actors, each interacting with a different component of the architecture.

- **User:** the primary actor of the system. The user interacts with the application to store, retrieve, share, and manage files within the distributed network. The user is also a participating node in the cluster, contributing storage capacity to the system.
- **Administrator:** responsible for monitoring and supervising the infrastructure. The administrator interacts exclusively with the monitoring interface to observe network activity, inspect cluster topology, track performance metrics, and monitor system health in real time. The administrator has no access to user data or file content.

4.5.2 User Requirements

The functional requirements associated with the user actor are the following:

- The storage of a file on the distributed network.
- The retrieval of a previously stored file.
- The deletion of a file, with a recycle bin and restore capability.
- The consultation of file information and availability status.
- The organization of files through a folder hierarchy.
- The search, sorting, and filtering of stored files.
- The control of ongoing transfers: pause, resume, cancel, and retry.

-
- The consultation of node status: connectivity health, storage contribution, node role, and battery impact.
 - The access to advanced diagnostics: event log, cluster member list, and relay server information.
 - The configuration of the device name and shared storage quota.

4.5.3 Administrator Requirements

The functional requirements associated with the administrator actor are the following:

- The consultation of the overall network health: relay server status, number of connected participants, active clusters, and churn rate.
- The visualization of the network topology through an interactive map of all connected devices and clusters.
- The consultation of device details: role, available storage, reliability score, address, and last activity.
- The consultation of cluster details: member count, aggregated storage, and average reliability score.
- The consultation of event counters and system logs in real time.
- The subscription to a live stream of routing events with source, destination, and volume.
- The consultation of fragment-to-device mapping: which fragments of a given file are stored on which device.
- The consultation of upload history: which user stored which file and the distribution of its fragments across the network.

4.6 Use Case Diagrams

Use case diagrams constitute the first step in UML analysis. They provide a global view of the functional behavior of a software system and answer the fundamental question: *who does what?* By capturing the behavior of a system as perceived from an external point of view, use case diagrams abstract the system's functionality from its technical realization, without focusing on internal implementation details. The following subsections present the use case diagrams elaborated for the system, covering its major functional domains.

4.6.1 General Use Case Diagram

Figure 4.3 presents the general use case diagram, offering a high-level overview of the main functionalities provided by the system from the end-user's perspective. It identifies the primary actors and their interactions with the core system features.

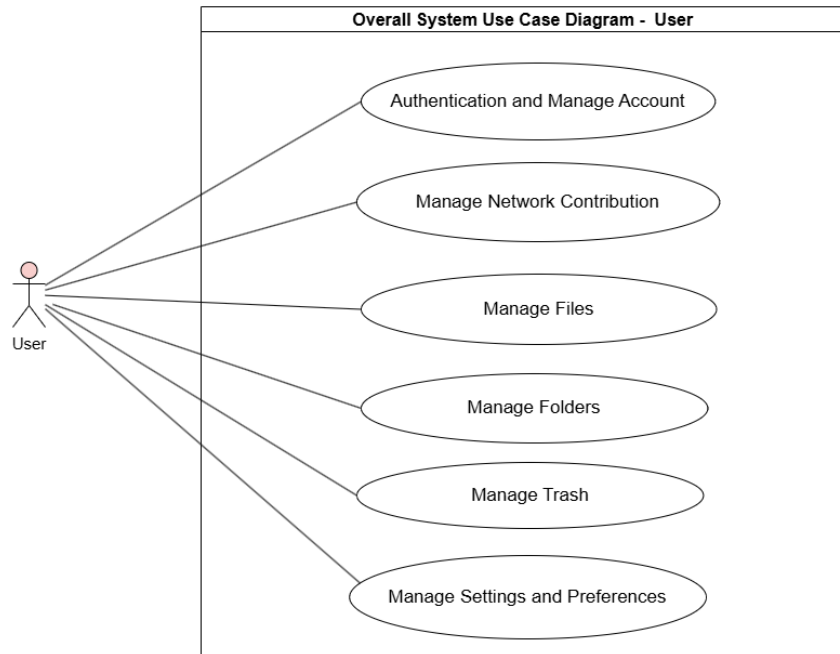


Figure 4.3: General Use Case Diagram of the System.

Beyond this general view, the functional scope of the system is decomposed into several modules. To preserve the readability of this chapter, the detailed use case diagram of each user-facing module is provided in Appendix 5.7:

- **Authentication and Account Management** : account registration, login, logout, profile editing, and password management (Figure A.1).
- **Manage Files** : storing, retrieving, renaming, moving, sharing, listing, and deleting files (Figure A.2).
- **Manage Folders**: folder creation, renaming, hierarchical organization, and directory-level access control (Figure A.3).
- **Manage Recycle Bin** — reviewing, restoring, or permanently deleting removed items before they are purged (Figure A.4).
- **Manage Settings and Preferences** :configuring storage quota, network restrictions, display theme, and security options (Figure A.5).

4.6.2 Administrator System

Figure A.6 presents the administrator use case diagram, which groups the supervision operations available through the monitoring interface, including network health monitoring, topology visualization, node and cluster inspection, and real-time event tracking.

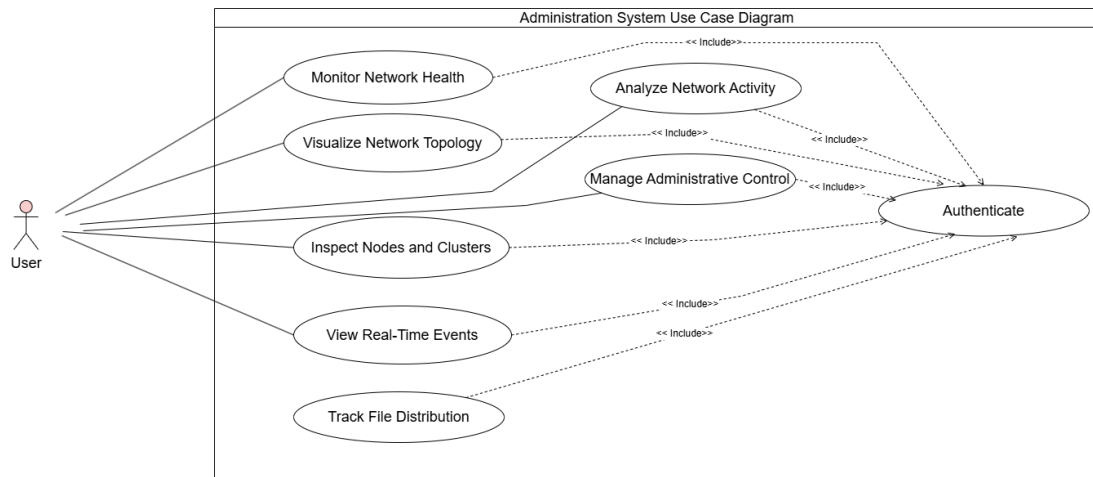


Figure 4.4: Use Case Diagram of Administrator System.

4.7 Class Diagram

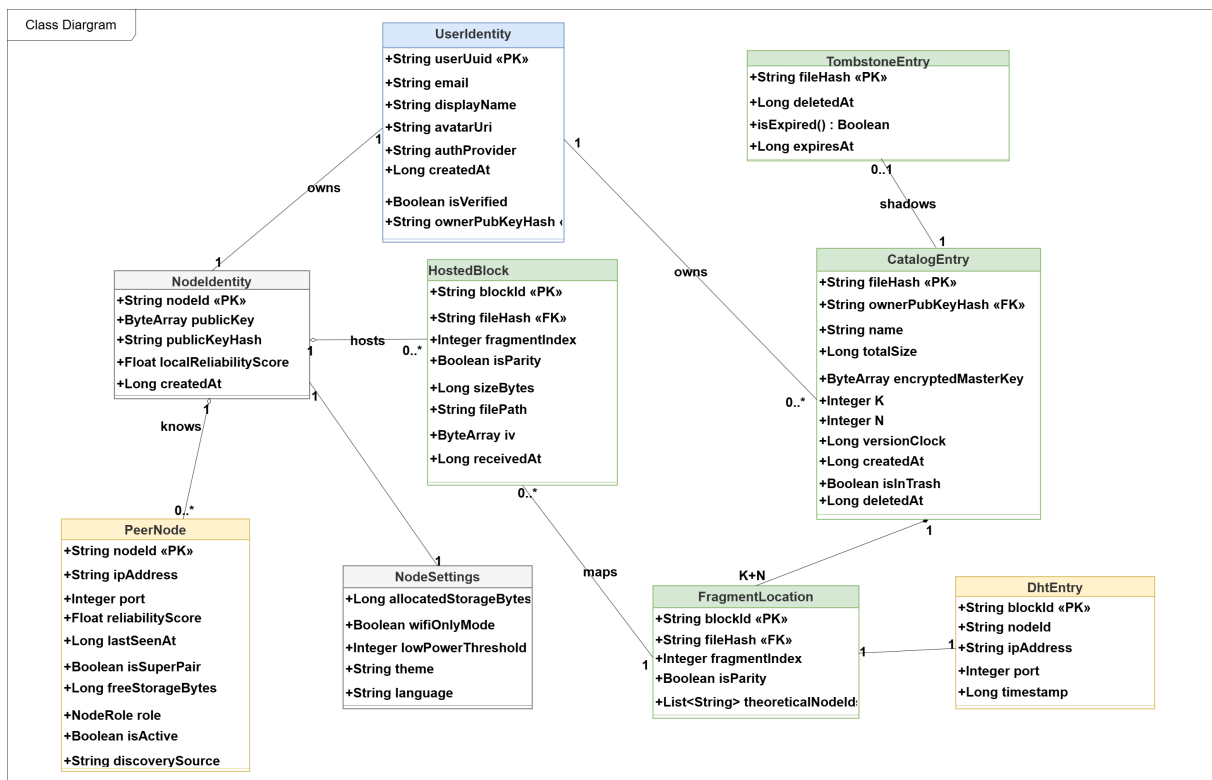


Figure 4.5: Class Diagram of the System.

Figure 4.5 presents the class diagram of the system. This static structural diagram captures the principal entities of the system, their attributes, methods, and the relationships.

4.8 Sequence Diagrams

Sequence diagrams belong to the family of behavioral interaction diagrams in UML. They describe the dynamic exchanges between actors and system components from a temporal perspective, answering the question: *how does the system behave?* For each scenario, a structured textual description (goal, pre-conditions, main flow, and exceptions) is provided in Appendix 5.7, while the corresponding sequence diagram is presented below.

4.8.1 Soft Delete File

The soft delete operation allows a user to logically remove a file from the application without immediately deleting its distributed fragments from the network. Instead of permanently erasing the metadata and blocks, the system marks the file as deleted using a tombstone mechanism. This approach enables future restoration while preserving consistency across the distributed environment. Figure 4.6 illustrates the sequence of interactions performed during the soft deletion process, including metadata updates, gossip propagation, and synchronization of the deletion state across the cluster.

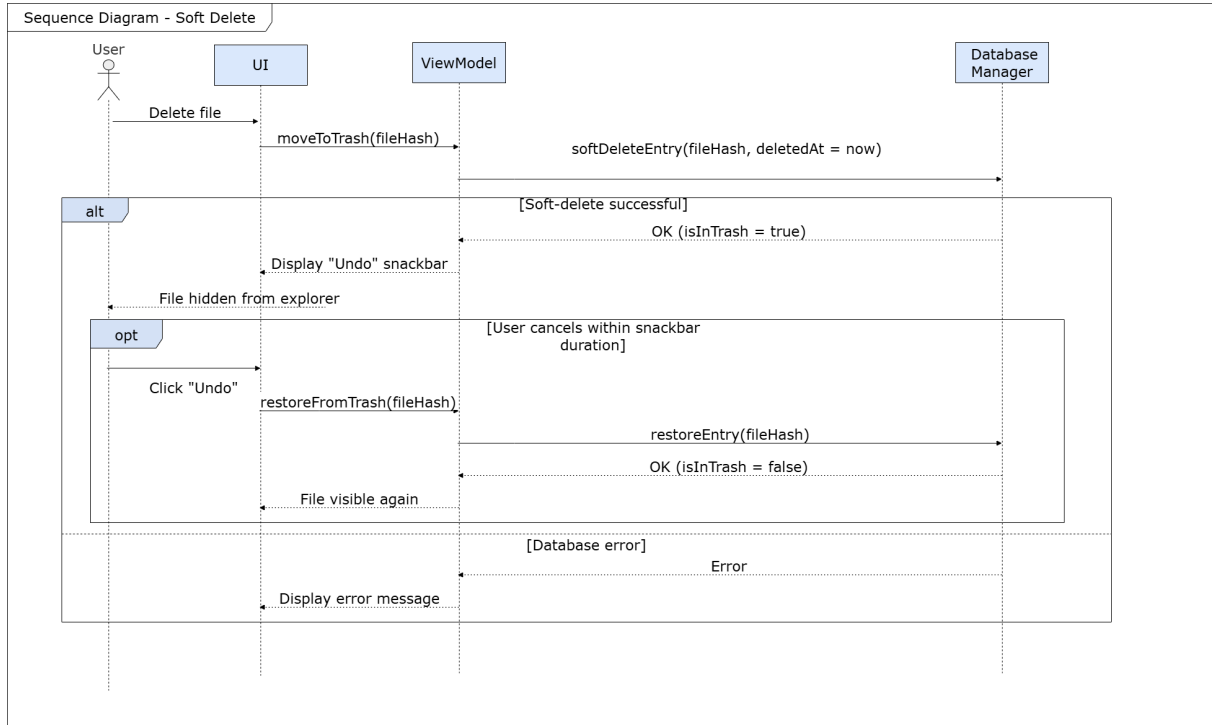


Figure 4.6: Sequence Diagram of Soft Delete File.

4.8.2 Permanently Delete File

The permanent deletion scenario describes the irreversible removal of a file from the network. Unlike the soft delete operation, this process eliminates both the metadata references and the physical encrypted fragments stored on the participating nodes. The operation ensures that the file can no longer be recovered or reconstructed. Figure 4.7 presents the sequence diagram associated with the permanent deletion workflow.

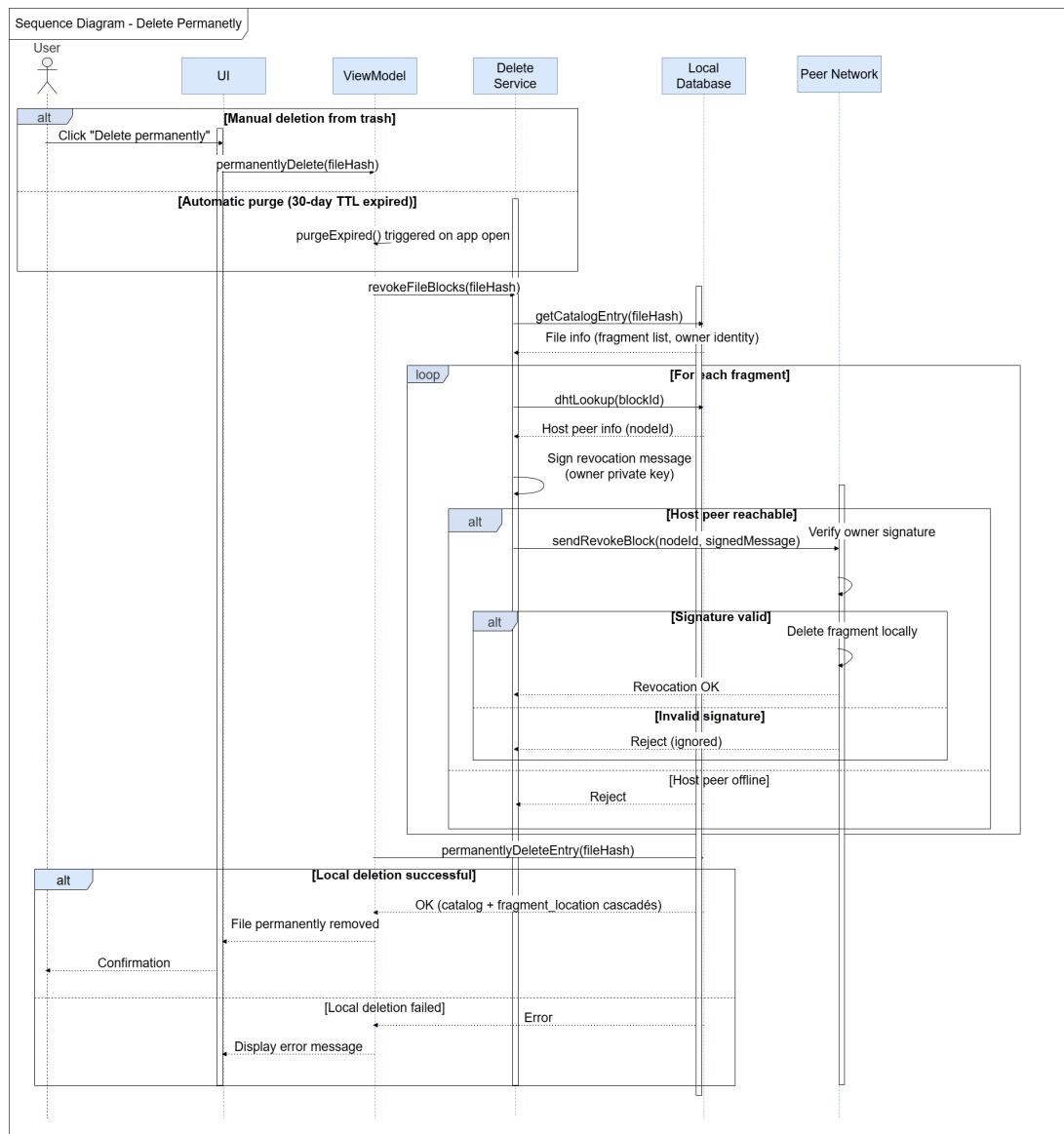


Figure 4.7: Sequence Diagram of Permanently Delete File.

4.8.3 File Distribution

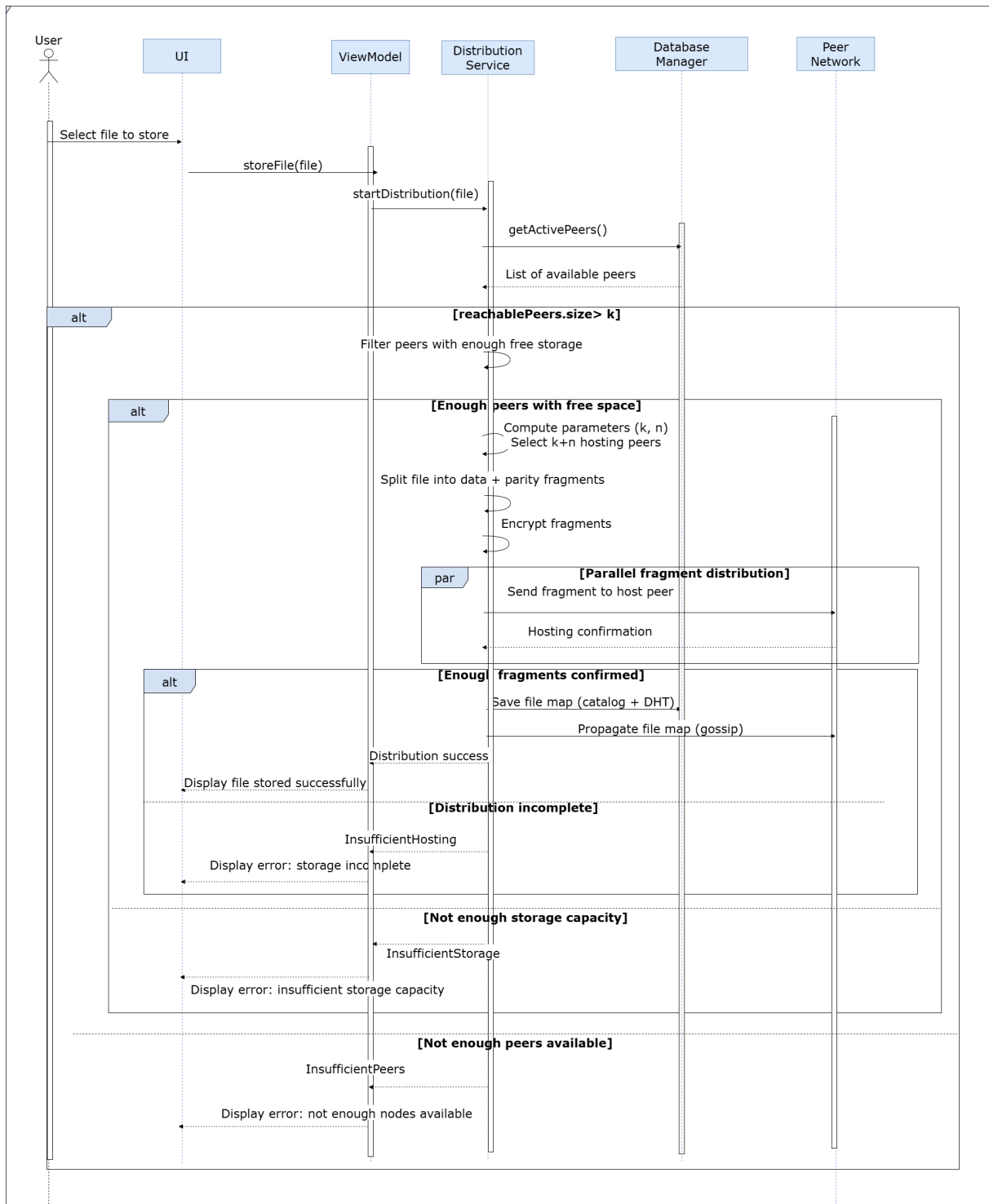


Figure 4.8: Sequence Diagram of File Storage

Figure 4.8 presents file distribution scenario describes the process by which a user stores and distributes a file across the network. Its detailed textual description is given in Table B.1 (Appendix 5.7).

4.8.4 File Retrieval

figure 4.9 presents the file retrieval scenario describes the process by which a user locates, downloads, decrypts, and reconstructs a file previously stored on the network. Its detailed textual description is given in Table B.2 (Appendix 5.7).

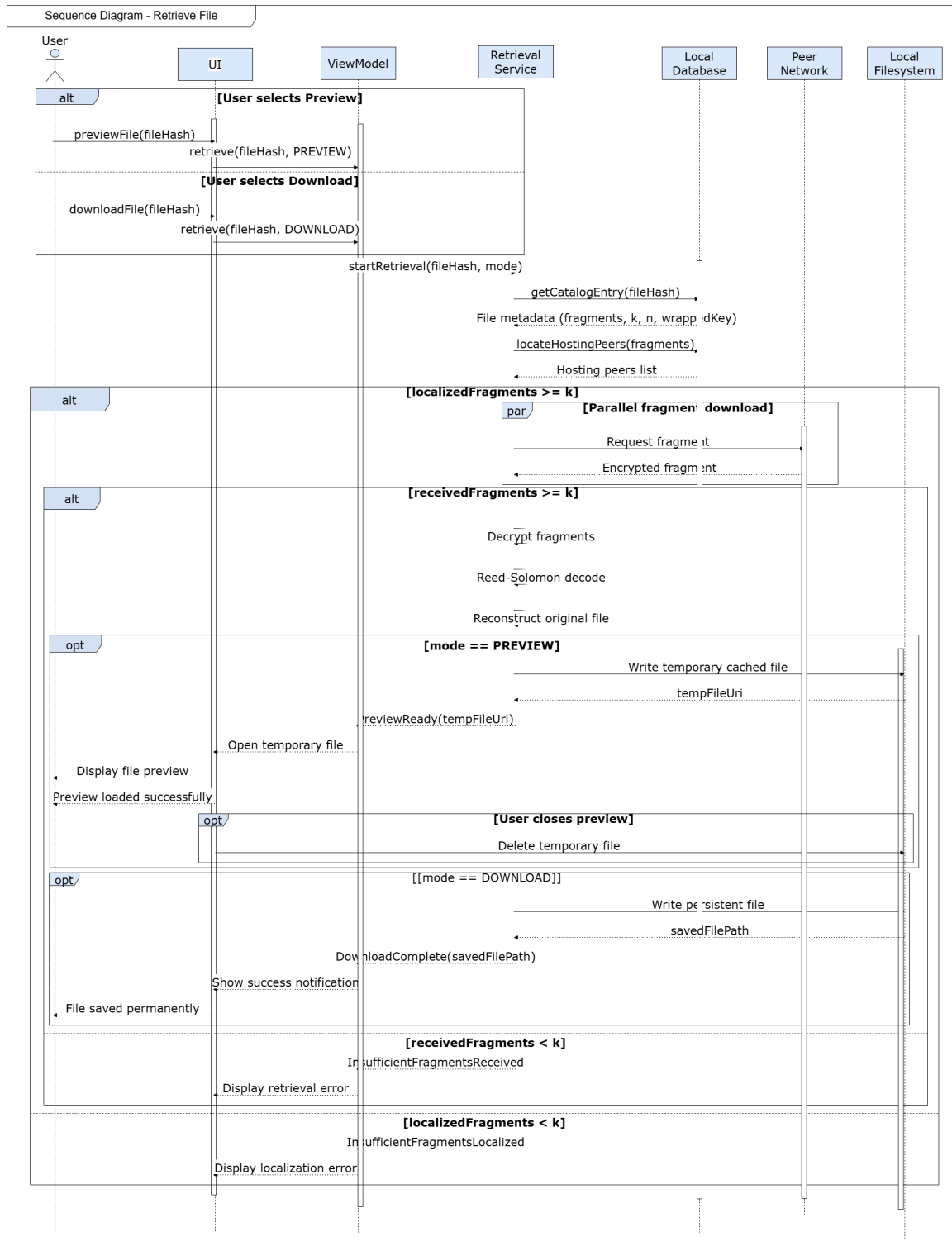


Figure 4.9: Sequence Diagram of the File Retrieval.

4.9 Activity Diagram

The activity diagram describes the workflow executed when a node attempts to join a cluster. It highlights the discovery process, communication with cluster-heads, validation steps, retry mechanisms, and the final integration of the node into the distributed environment. Figure 4.10 presents the detailed activity flow of the cluster join procedure.

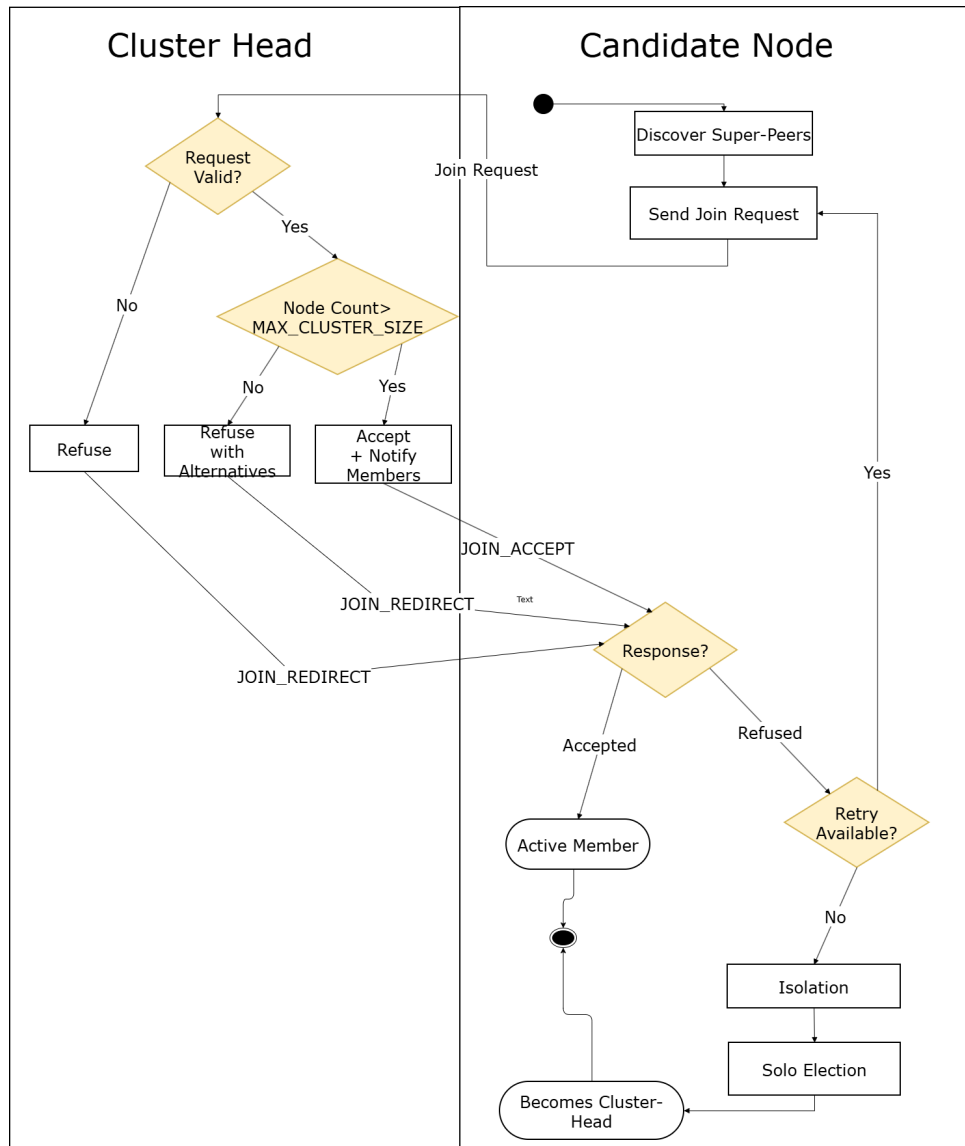


Figure 4.10: Activity Diagram of Cluster Join.

4.10 State Machine Diagram

The state machine diagram models the lifecycle of a node inside the system. It describes the different states through which a node passes, from its initial discovery and cluster integration to its role as a member or cluster head. The transitions between states represent the events and operations that trigger changes in the node's status throughout its lifecycle. Figure 4.11

illustrates the complete state machine of the node within the distributed storage environment.

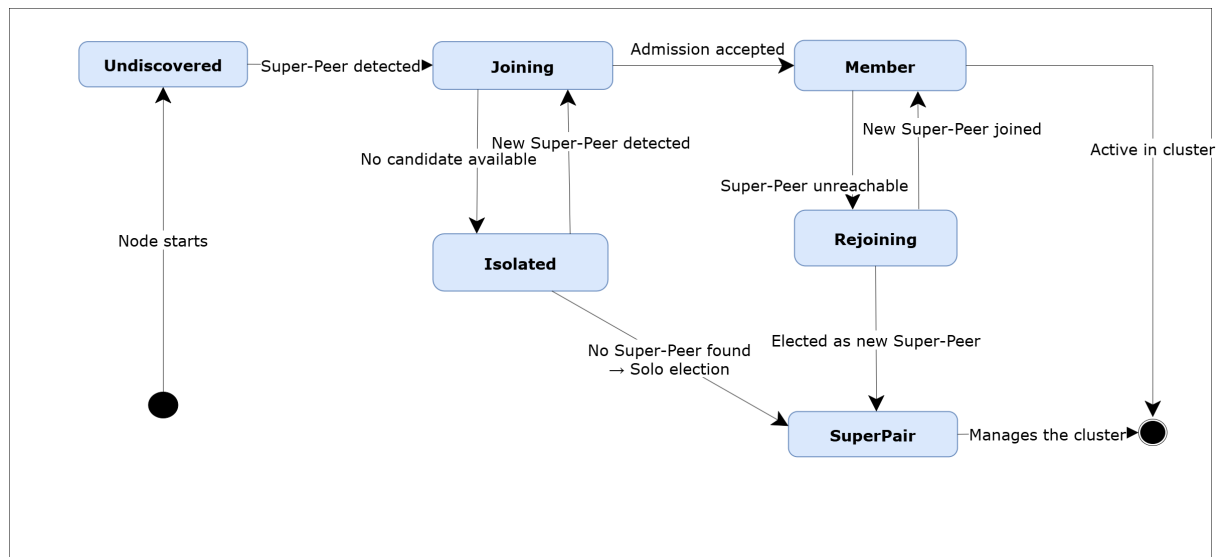


Figure 4.11: State Machine Diagram of the Node.

5 Implementation

5.1 Introduction

The exponential growth of data generated by mobile devices has made decentralised storage one of the central challenges of modern Big Data engineering. Rather than relying on a centralised cloud infrastructure, this approach proposes to transform a set of ordinary smartphones into a cooperative distributed storage cluster, applying at mobile scale the same fault-tolerance principles that underpin large-scale Big Data systems such as HDFS, Ceph, and Azure Data Lake Storage.

Having completed the design of our system, we present in this final chapter the implementation and deployment of the mobile application. First, we enumerate and describe all the software tools that enabled us to bring our work to fruition. We then outline the security mechanisms put in place to guarantee data confidentiality. Next, we present the hardware environment used for testing along with the results obtained. Finally, a concrete view of the application is illustrated through screenshots representing its most important interfaces.

5.2 Development Methodology

The development process adopted in this project is inspired by the **Agile methodology**, an iterative and incremental approach widely used in software engineering. Rather than delivering the entire system in a single phase, Agile organizes the work into successive cycles called Sprints, each producing a functional and testable increment of the system.

The process revolves around four recurring phases. The first phase consists of **requirements analysis**, during which the system's objectives, functionalities, and constraints are identified and formalized as *User Stories*. The second phase produces the *Product Backlog*, a prioritized list of all features, tasks, and improvements to be implemented. The third phase is **Sprint Planning**, where the team selects the tasks to be completed during the upcoming Sprint, estimates their complexity, and organizes the work. The fourth phase is the **Sprint development** itself, during which coding, testing, integration, and bug fixing are carried out in an iterative manner.

At the end of each **Sprint**, the delivered increment is reviewed and the backlog is updated to reflect progress made and any new requirements identified. This cycle then repeats until the system is complete, ensuring continuous delivery of working software, early detection of issues, and the ability to adapt to evolving requirements throughout the project.

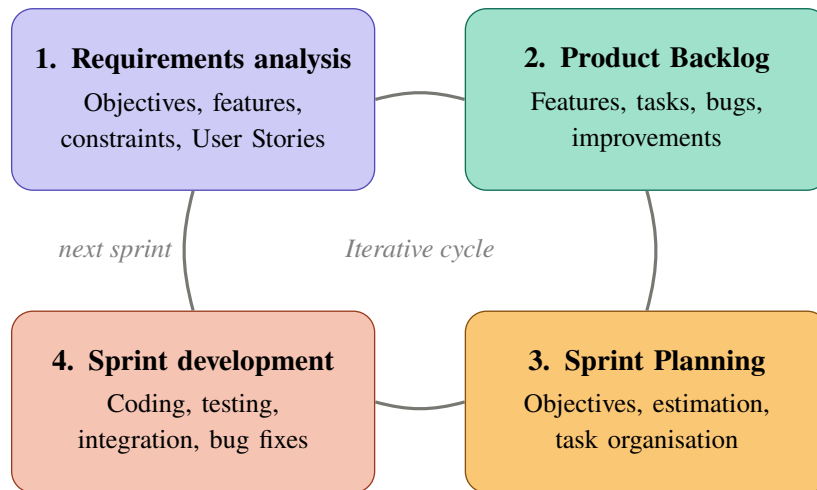


Figure 5.1: Agile iterative development cycle.

5.3 Development Environment

The implementation of a distributed storage system at mobile scale requires a technology stack that combines reliability, performance, and native support for concurrent and networked operations, qualities that are equally central in traditional Big Data engineering. The tools and frameworks selected for the system are presented below.

5.3.1 Android Studio



Figure 5.2: Android Studio

Android Studio is the official Integrated Development Environment (IDE) for the Android platform [45]. Developed by Google and built on IntelliJ IDEA, it provides a complete set of tools for designing, developing, testing, and deploying Android applications.

Android Studio incorporates an advanced Android emulator, a visual layout editor, performance profiling tools, and native support for the Gradle build system. It supports both Kotlin and Java, and offers intelligent code completion along with an integrated debugger.

We chose Android Studio because of its native integration with the Android SDK, its first-class Kotlin support, and the comprehensive toolchain it provides for building, testing, and deploying the application.

5.3.2 Kotlin



Figure 5.3: Kotlin

Kotlin is a modern, statically typed programming language developed by JetBrains [46]. It became the officially recommended language for Android development by Google in 2019. Kotlin is fully interoperable with Java and runs on the Java Virtual Machine (JVM).

Kotlin offers a concise and expressive syntax that significantly reduces the amount of boilerplate code compared to Java. It natively supports nullability handling, coroutines for asynchronous programming, extension functions, and data classes. These features produce code that is safer, more readable, and more maintainable.

We chose Kotlin for its native coroutines support, which allowed us to manage network and storage operations in an asynchronous, non-blocking manner, an essential requirement in a distributed storage system where multiple data streams must be handled concurrently without blocking the main thread.

5.3.3 Jetpack Compose



Figure 5.4: Jetpack Compose

Jetpack Compose is Google’s modern UI toolkit for building native Android interfaces [47]. It adopts a declarative paradigm that simplifies and accelerates UI development by allowing the interface to be described as a function of the application state.

Unlike the traditional XML-based approach, Jetpack Compose allows the entire user interface to be defined directly in Kotlin. The UI automatically updates whenever the application state changes, a process known as recomposition. Compose integrates natively with other Jetpack components such as ViewModel, Navigation, and LiveData.

Choosing Jetpack Compose allowed us to build reactive, modern interfaces following Material Design 3 guidelines, while maintaining a unified and consistent code base across all screens of the application.

5.3.4 Hilt

Hilt is a dependency injection framework for Android developed by Google, built on top of Dagger [48]. It simplifies dependency injection in Android applications by providing standardized containers for each Android component.

Dependency injection is a design pattern that decouples the components of an application by supplying their dependencies from outside rather than having them instantiate those dependencies themselves. Hilt automates this management by generating the required wiring code at compile time, eliminating runtime errors and improving the testability of the application.

We used Hilt to manage dependencies between the various layers of the system architecture, in particular between use cases, repositories, and network services, in accordance with the Clean Architecture adopted in this project.

5.3.5 Room Database



Figure 5.5: Room Database

Room is a data persistence library developed by Google as part of the Jetpack suite [49]. It provides an abstraction layer over SQLite, enabling access to the local database in a more fluid manner while retaining the full power of SQLite.

Room consists of three main components: entities, which represent database tables; DAOs (Data Access Objects), which define the data access methods; and the Database class, which serves as the main access point to the persistent database. Room validates SQL queries at compile time, preventing runtime errors.

We chose Room to locally persist the distributed file catalogue, fragment metadata, and cluster member information. This local persistence layer plays a role analogous to that of a local data node cache in a Hadoop cluster: it guarantees data availability and query speed even when network connectivity is lost.

5.3.6 Ktor



Figure 5.6: Ktor

Ktor is an asynchronous framework for building server and client applications in Kotlin, developed by JetBrains [50]. It is built on Kotlin coroutines and provides a clean, intuitive API for constructing web services and WebSocket servers.

Ktor follows a modular architecture in which each feature is provided as a plugin. It natively supports WebSockets, HTTP routing, JSON serialisation, and TLS connections. Its lightweight footprint and high performance make it well suited for embedded servers and microservices.

In our system, Ktor was used to develop the WebSocket relay server responsible for signalling and message transport between nodes located behind separate NATs. This server fulfils a role comparable to that of a coordinator node in a distributed system: it routes control messages between clusters and enables devices on 4G and Wi-Fi networks to communicate transparently. The relay server was deployed on Render (Frankfurt, Germany), ensuring low-latency connectivity for our test devices.

5.3.7 Android NDK and C++



Figure 5.7: Android NDK and C++

The Android NDK (Native Development Kit) is a set of tools that allows native code written in C or C++ to be embedded in an Android application [51]. It is particularly suited to processing tasks that require high performance, such as signal processing, low-level cryptography, and coding algorithms.

The NDK uses the Java Native Interface (JNI) as a bridge between Kotlin/Java code and native C++ code. Native libraries are compiled via CMake into .so files that are loaded dynamically at application startup. This approach grants access to low-level system resources while benefiting from the execution speed of native code.

We used the NDK to implement the Reed-Solomon erasure coding algorithm directly in C++. Erasure coding is one of the fundamental data redundancy techniques in Big Data storage systems, it is notably used by Google Colossus, Facebook f4, and Apache HDFS Erasure Coding, because it provides strong fault tolerance at a much lower storage overhead than simple replication. Our implementation uses a $(K = 4, N = 2)$ configuration: each file is divided into 4 data fragments and 2 parity fragments, for a total of 6 fragments. The original file can be reconstructed from any 4 of the 6 fragments, tolerating the simultaneous loss of any 2 nodes. Implementing this algorithm in native C++ rather than Kotlin was necessary to meet the real-time performance requirements of fragmentation and reconstruction on resource-constrained mobile hardware.

5.3.8 React

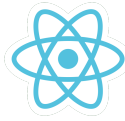


Figure 5.8: React

React is an open-source JavaScript library developed by Meta for building interactive user interfaces [52]. It is based on a component-driven architecture in which each element of the interface is an autonomous, reusable unit.

In our solution, React was used to develop the relay server administration dashboard. This web interface provides a real-time view of the relay server’s activity (active WebSocket connections, connected nodes, traffic metrics, and server health indicators) and served as our primary observability tool throughout all test sessions.

5.4 Core Modules Implementation

Security

Security and data confidentiality are critical concerns in any distributed storage system — all the more so in a fully decentralized architecture where, unlike a conventional cloud data centre, there is no single trusted operator controlling the storage nodes. In our system, user data transits through and is stored on devices owned by third parties, which imposes strict security mechanisms at every level of the system. This constraint is structurally similar to the one faced by Big Data systems that store sensitive data across heterogeneous commodity nodes in multi-tenant environments.

We focused on three fundamental aspects of security: node identity, fragment encryption, and secure key distribution.

5.4.1 Node Identity

To ensure the authenticity of each participant in the network and prevent any identity spoofing, every device holds a unique cryptographic identity generated and protected by the Android Keystore system.

The Android Keystore is a secure storage system for cryptographic keys. On devices equipped with a hardware security module (TEE, Trusted Execution Environment), private keys never leave the hardware and cannot be extracted even by an application with elevated privileges. This hardware-backed protection ensures that a node’s identity is irrevocably bound to the physical device that hosts it.

5.4.1.1 Identity Generation

At first launch, the application generates an asymmetric EC (Elliptic Curve) key pair on the secp256r1 (P-256) curve directly inside the Keystore. The public key constitutes the node's permanent identifier in the network. Every message sent by a node is digitally signed with its private key, allowing other nodes to verify its authenticity before any processing takes place.

5.4.1.2 Node Verification

Upon receiving a message, the destination node verifies the sender's digital signature against its known public key. Any message with an invalid signature is immediately discarded. This mechanism prevents the injection of forged messages into the network and guarantees the integrity of all communications between nodes.

5.4.2 Fragment Encryption

To ensure the confidentiality of data stored at node devices, every file fragment is individually encrypted before being transmitted or stored on a third-party device. The algorithm used is AES (Advanced Encryption Standard) in GCM (Galois/Counter Mode) with 256-bit keys.

AES is the most widely adopted symmetric encryption standard. GCM mode combines stream encryption with data authentication, guaranteeing both the confidentiality and the integrity of each fragment. Each encryption operation uses a unique randomly generated 96-bit Initialisation Vector (IV), making it impossible to infer any information by comparing ciphertexts.

Among the advantages of the AES-GCM algorithm, we highlight:

- A high security level (256 bits) recognised by NIST as the US federal encryption standard;
- Integrated data authentication via the GCM tag, which detects any alteration of the fragment before decryption;
- Excellent execution performance thanks to hardware AES acceleration present in modern ARM processors.

5.4.3 Key Distribution

Secure distribution of encryption keys between nodes is a fundamental challenge in decentralised systems. we uses a key exchange protocol based on ECDH (Elliptic Curve Diffie-Hellman) combined with HKDF (HMAC-based Key Derivation Function) to establish shared keys without any secret ever being transmitted in plain text over the network.

ECDH allows two parties to derive a shared secret from their respective public keys without any prior communication of a secret value. This shared secret is then passed to HKDF, which produces a derived AES-256 key that is unique to the communicating pair of devices.

This approach guarantees Perfect Forward Secrecy: even if a private key is compromised at a later date, past sessions remain protected.

5.5 Testing

In order to validate the implementation of our work, we tested it in an environment representative of real-world usage conditions. A minimum of six storage nodes is required to distribute the six fragments produced by the ($K = 4$, $N = 2$) Reed-Solomon scheme — one fragment per node. Our cluster was composed of one cluster head coordinator and six regular member devices, for a total of seven smartphones.

5.5.1 Hardware Setup

The test cluster is composed of seven Android smartphones and one development laptop. Table 5.1 presents the specifications of each device.

Device	Processor	RAM	Storage	Android	Network
Oppo Reno 12 (CH)	Dimensity 7300E Octa-core 2.0 GHz	8 GB	256 GB	14	Wi-Fi ac / 4G
Oppo F9	Helio P60 Octa-core 2.0 GHz	6 GB	128 GB	10	Wi-Fi n / 4G
Samsung A11	Helio G25 Octa-core 2.0 GHz	3 GB	32 GB	11	Wi-Fi n / 4G
Benco V80s	Helio A22 Quad-core 2.0 GHz	4 GB	64 GB	12	Wi-Fi n / 4G
Samsung M17	Snapdragon 439 Octa-core 2.0 GHz	4 GB	64 GB	11	Wi-Fi n / 4G
Xiaomi Redmi Note 10	Snapdragon 678 Octa-core 2.2 GHz	4 GB	64 GB	11	Wi-Fi ac / 4G
Infinix Hot 11S	Helio G88 Octa-core 2.0 GHz	4 GB	64 GB	11	Wi-Fi n / 4G
HP Omen Transcend 14 (Dev)	Intel Core Ultra 7 RTX 5050	16 GB	–	Windows 11	–

Tableau 5.1: Hardware specifications of the test cluster devices.

The network infrastructure used during testing comprised two connectivity environments.

- A home Wi-Fi network (802.11ac, 100 Mbps throughput) served as the primary local connectivity medium for most devices.

- A 4G LTE mobile connection (Djezzy operator, average downlink speed of 15 Mbps) was used to simulate wide-area network conditions for selected test scenarios.
- The relay server was deployed on Render (Frankfurt, Germany) and served as the single fixed infrastructure point for all inter-device signaling.

5.5.2 Network Topology

Figure 5.9 illustrates the physical network topology used to carry out the tests.

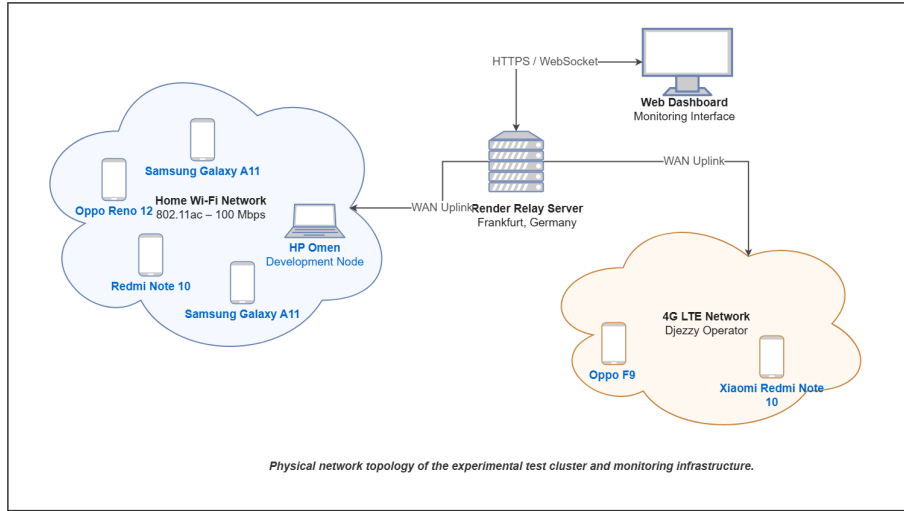


Figure 5.9: Physical network topology of the experimental test cluster and monitoring infrastructure.

5.5.3 Performance Evaluation by Simulation

To evaluate our proposed solution’s protocols beyond the 7-device physical cluster, we used **PeerSim 1.0.5** to simulate networks of up to 10 000 nodes (fixed seed 1234567890, 1 cycle = 2 s simulated). We focused on three properties directly relevant to Big Data distributed storage: gossip scalability, coordinator failover, and churn resilience.

5.5.3.1 DHT Gossip Convergence and Scalability

Table 5.2 and Figure 5.10 show that the DHT achieves $O(\log N)$ **convergence**: 100 nodes converge in 2 cycles, 10 000 nodes in 5 cycles (10 s). This matches the anti-entropy gossip property of Apache Cassandra and HDFS block-report propagation.

Network size	Cycle to 99% convergence	Time (simulated)	Nodes converged at cycle 3
100	2	4 s	100 (100%)
500	3	6 s	500 (100%)
1 000	3	6 s	991 (99.1%)
5 000	4	8 s	4 134 (82.7%)
10 000	5	10 s	9 157 (91.6%)

Tableau 5.2: DHT gossip convergence time as a function of network size (fan-out = 2, 100 blocks).

Figure 5.10 illustrates the convergence curves for all five network sizes. The data confirms an $O(\log N)$ **convergence** property: even at 10 000 nodes, the entire DHT converges in at most 5 gossip cycles (10 simulated seconds). This epidemic dissemination behaviour is structurally identical to the anti-entropy gossip employed by Apache Cassandra to propagate ring membership information, or by HDFS to synchronise block reports across DataNodes — both systems rely on the same $O(\log N)$ spreading guarantee to achieve cluster-wide consistency without centralised coordination.

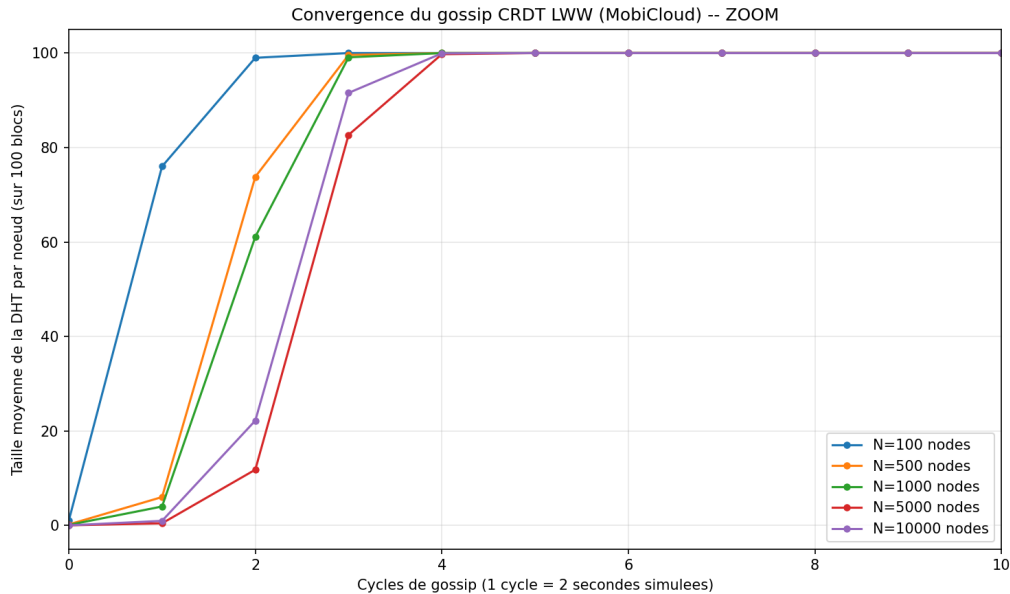


Figure 5.10: DHT gossip convergence curves (CRDT LWW) for $N \in \{100, 500, 1000, 5000, 10000\}$ nodes with fan-out = 2. The y-axis shows the mean DHT size per node (out of 100 blocks).

5.5.3.2 Coordinator Re-election After Failure

We simulated the sudden failure of all Cluster-heads at cycle 20 ($N = 1\,000$). As shown in Figure 5.11, the active Cluster-head count drops from 82 to 0, then recovers to 85 within **one cycle (2 s)** — with no external coordinator, analogous to Kafka’s KRaft leaderless election.

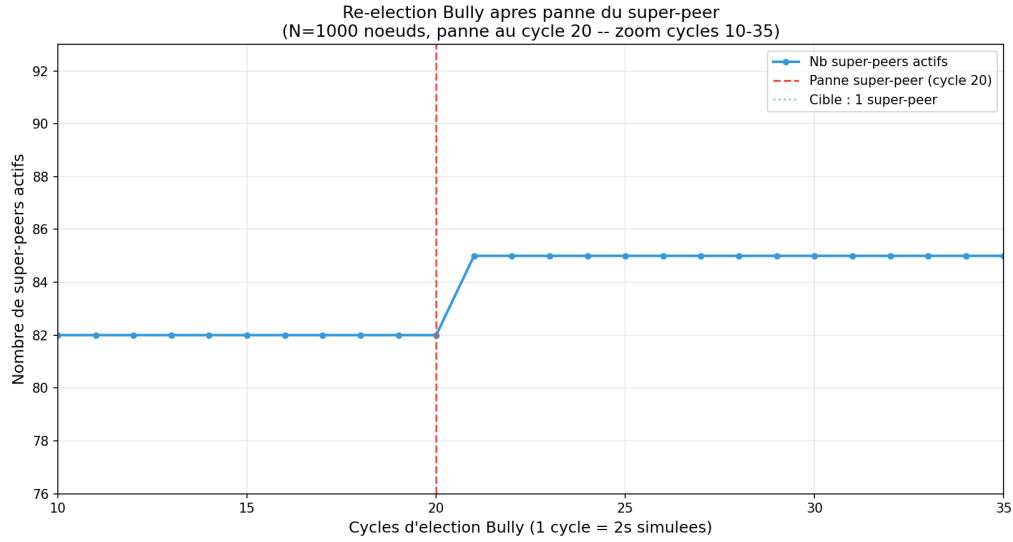


Figure 5.11: Bully re-election after Cluster-head failure at cycle 20 ($N = 1000$ nodes). The network recovers a new stable Cluster-head set within one election cycle (2 simulated seconds).

5.5.3.3 DHT Robustness Under Mobile Churn

We tested churn rates of 0%, 10%, 20%, and 30% ($N = 1000$, fan-out = 2). As shown in Figure 5.12, the protocol stays **above 95% availability** even at 30% churn — a failure rate far more severe than typical HDFS DataNode churn — and re-converges within 1–2 cycles after each departure wave.

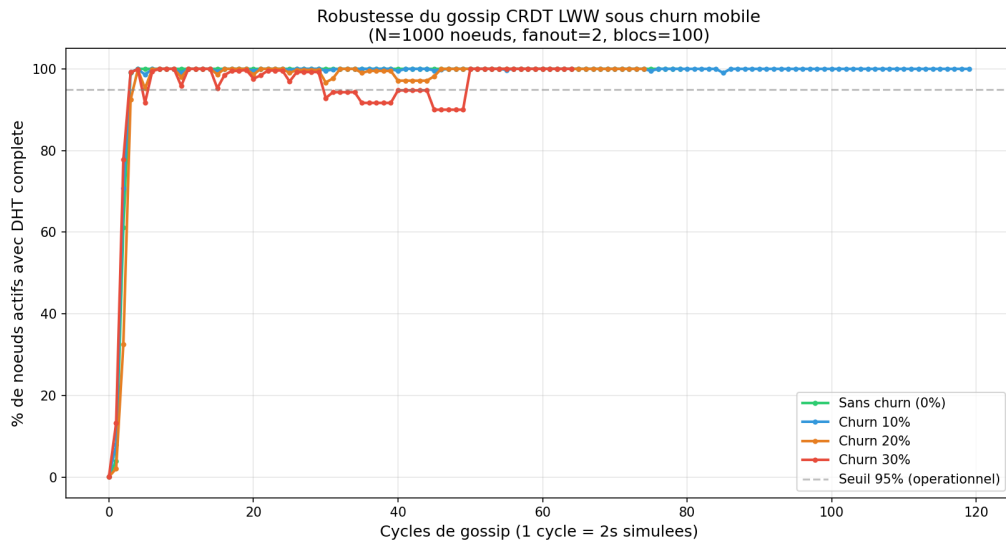


Figure 5.12: DHT convergence robustness under mobile churn (0%, 10%, 20%, 30%) for $N = 1000$ nodes with fan-out = 2. The dashed line marks the 95% operational threshold.

5.5.4 Real Tests

We tested file distribution and retrieval for files of varying sizes across the seven-device cluster (one cluster head + six regular members). Each test distributes a file as 4 data fragments

and 2 parity fragments across the 6 regular member nodes, one fragment per node. Since all communication in MobiCloud is routed through the single Render relay server hosted in Frankfurt, the relay latency is a constant factor regardless of whether the communicating nodes belong to the same cluster or different clusters. The variable that determines transfer time is therefore exclusively the upload bandwidth of the sending device’s network connection. The results are presented in Table 5.3.

	File size	Network condition	Distribution time	Retrieval time
File 1	1 MB	Wi-Fi 802.11ac (all nodes)	1.4 s	1.1 s
File 2	10 MB	Wi-Fi 802.11ac (all nodes)	5.3 s	4.0 s
File 3	50 MB	4G LTE, Djezzy (source node)	42 s	47 s

Tableau 5.3: Summary of test results.

The results show consistent and predictable performance. For Files 1 and 2, all nodes were connected via Wi-Fi, providing sufficient upload bandwidth to the Frankfurt relay. For File 3, the source device was connected via 4G LTE (Djezzy), whose uplink speed averaged 5 Mbps during the test; this bandwidth constraint alone explains the higher transfer time, since the relay server itself introduced no additional bottleneck. This behaviour is consistent with the I/O-bound patterns observed in Big Data pipelines that rely on wide-area network links, where the bottleneck is always the last-mile upload capacity rather than the processing or routing infrastructure.

We also tested cluster resilience against cluster head failure. After forcibly stopping the coordinator node (Oppo Reno 12), a new cluster head was elected by the Bully algorithm in an average time of 97 seconds, which is consistent with the configured parameters (a 90-second timeout followed by the exchange of election messages). During the entire failover period, all 6 fragments remained accessible on the member nodes and file retrieval continued uninterrupted, confirming the fault-tolerance guarantee of the ($K = 4$, $N = 2$) erasure coding scheme.

5.6 Application Overview

In this section, we present the main interfaces of the application. The application is organised around a bottom navigation bar with four tabs: **Dashboard**, **Explorer**, **Community**, and **Settings**.

5.6.1 Tab 1: Dashboard

The Dashboard tab (Figure 5.13) displays the node’s current role (cluster head or member), its reliability score, storage contribution, and a real-time diagnostic log. A health banner indicates the overall cluster connectivity status.

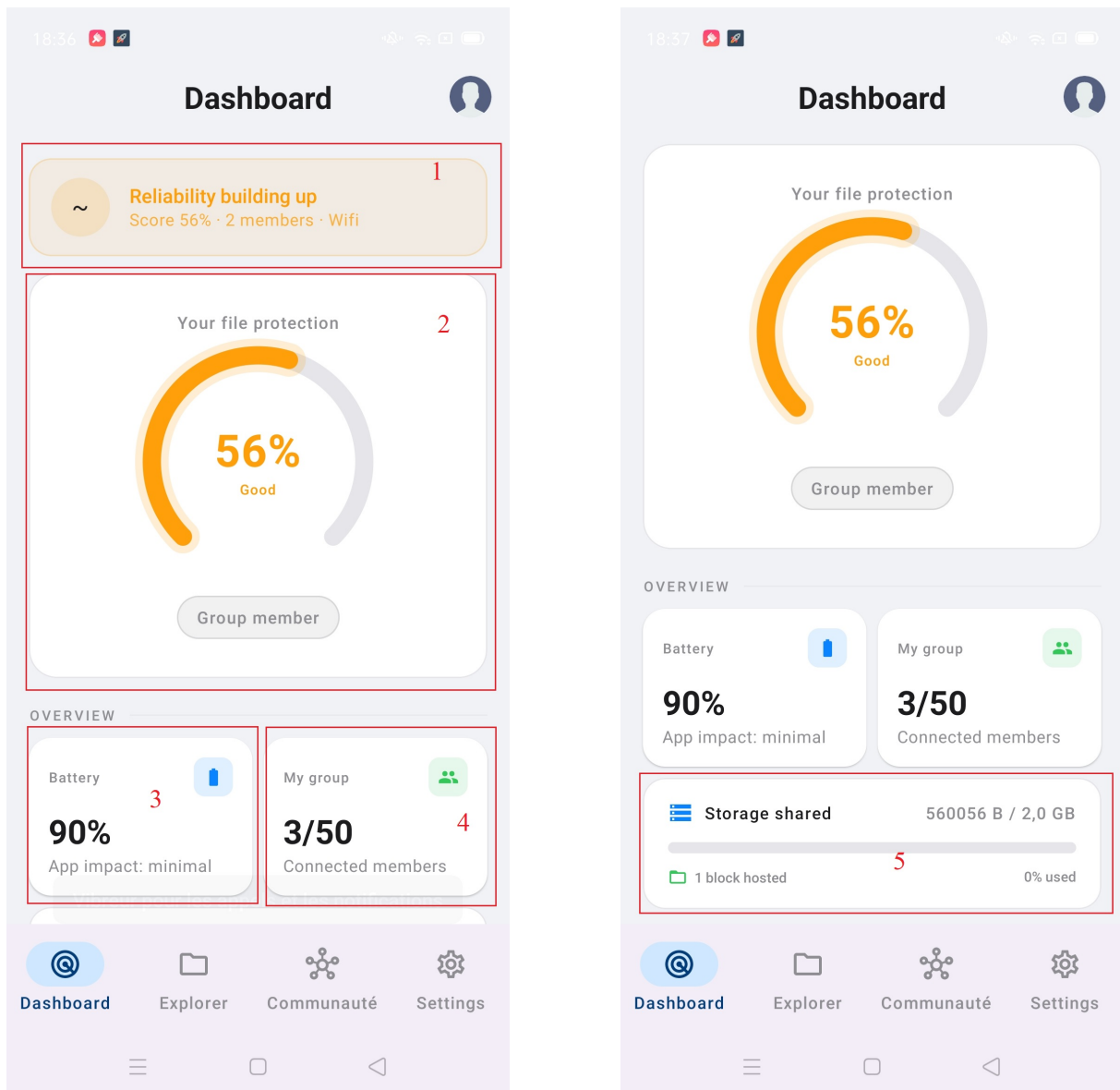


Figure 5.13: Dashboard tab: node role, reliability score, storage contribution, and connectivity health banner.

5.6.2 Tab 2: Explorer

The Explorer tab (Figure 5.14) lists all files stored in the distributed system with their name, size, and availability status. The floating action button opens the file picker to initiate a distribution. (1) "New Folder" button for folder creation along with the folder section, (2) file list with name, size, and replication status, (3) Upload file button (4) file retrieval success notification with completion badge

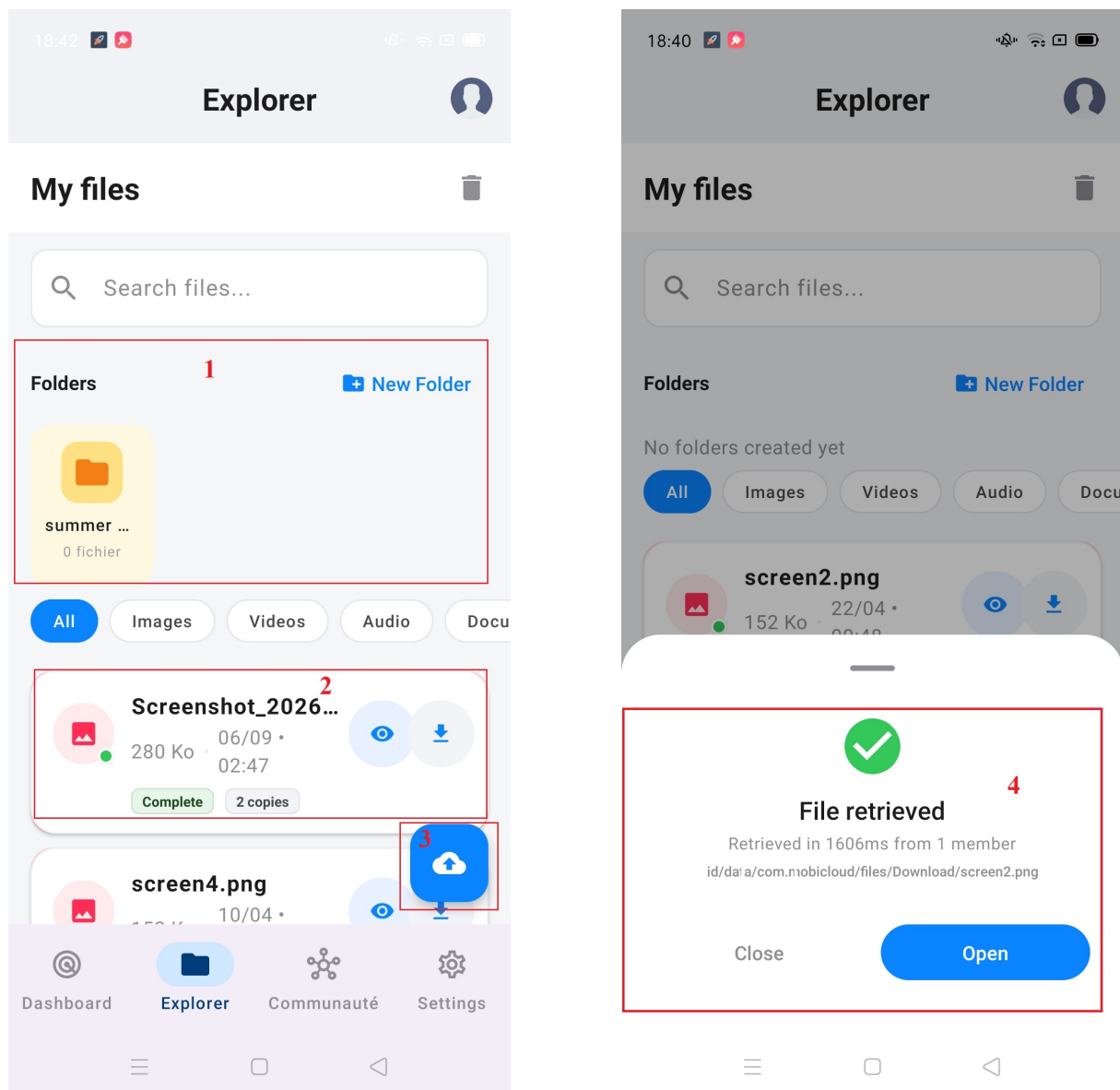


Figure 5.14: Explorer.

5.6.3 Tab 3: Community

The Community tab (Figure 5.15) shows the live member registry of the cluster. Each entry displays the node's truncated `nodeId`, its role (organizer or member), its reliability score as a percentage, and its connectivity status (Active, Degraded, or Offline). The cluster head is highlighted at the top with a distinct icon. Tapping a member opens a detail sheet showing the full `nodeId`, reliability score, battery level, communication channel, and time since the last heartbeat.

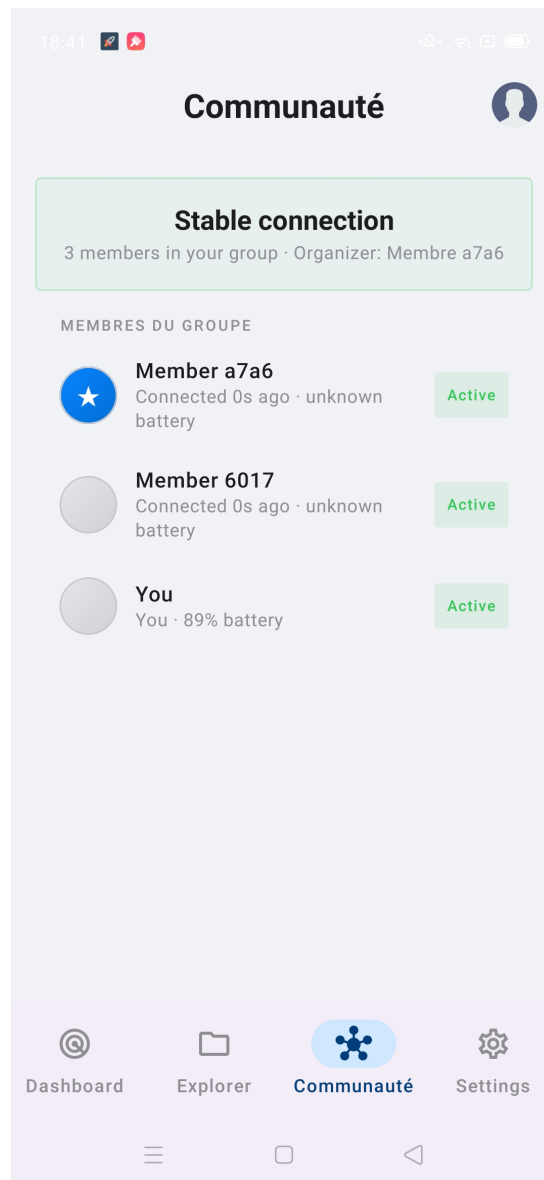


Figure 5.15: Community tab: real-time member registry, cluster topology, and heartbeat status.

5.6.4 Tab 4: Settings

The Settings tab (Figure 5.16) lets the user configure the storage quota allocated to system. A slider sets the amount of local space shared with the cluster, and a usage indicator shows how much of that quota is currently occupied by hosted fragments.

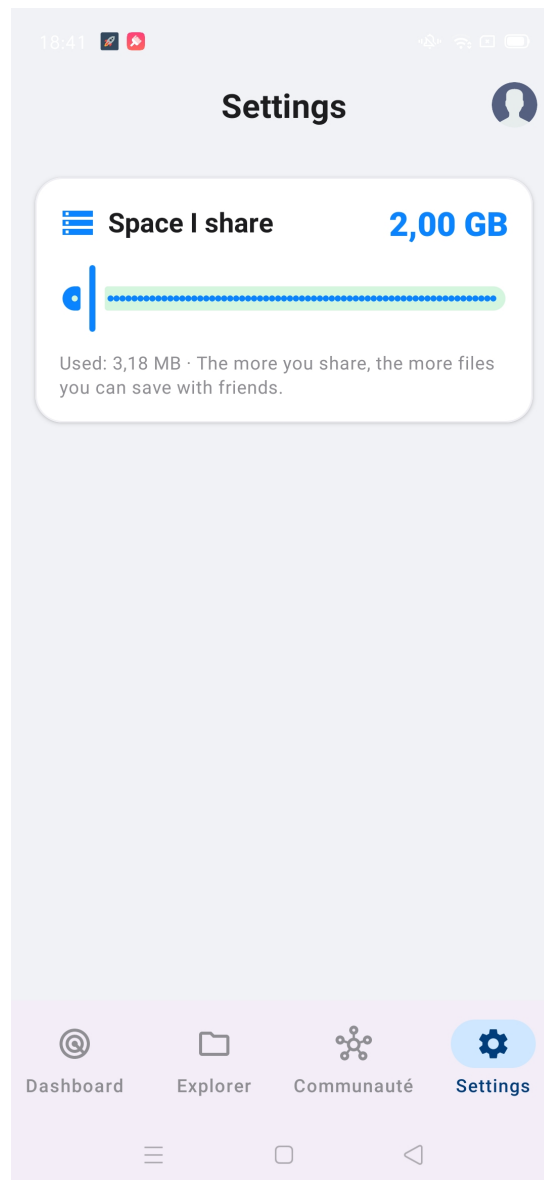


Figure 5.16: Settings storage quota configuration.

5.6.5 Administration Dashboard

In addition to the mobile application, a web-based administration dashboard was developed in React to supervise the relay server deployed on Render (Frankfurt). It provides a real-time view of the network, relay connectivity, active sessions and node count, churn rate, an interactive cluster topology graph, per-file fragment availability, and a live event log and served as our primary observability tool throughout all test sessions. As this dashboard is a monitoring and debugging instrument rather than a core contribution of the system, its detailed interface is presented in Appendix 5.7.

5.7 Conclusion

Throughout this chapter, we have presented the various development tools that enabled us to deliver the application. The chosen technologies — Kotlin, Jetpack Compose, Hilt, Room, Ktor, the Android NDK, and React — form a coherent and modern stack well suited to the demands of a distributed storage system operating at mobile scale.

The security mechanisms put in place, namely cryptographic identity via Android Keystore, AES-256-GCM fragment-level encryption, and key distribution via ECDH-HKDF, ensure the confidentiality and integrity of user data even when it is stored on third-party devices.

The tests carried out on a real cluster of seven smartphones validated the core features of the system: automatic node discovery, fault-tolerant file distribution using ($K = 4$, $N = 2$) Reed-Solomon erasure coding, and cluster resilience in the face of cluster head failure. From a Big Data engineering perspective, Our solution demonstrates that distributed storage principles (horizontal scalability across commodity nodes, erasure-coded redundancy, and decentralized coordination) can be applied effectively on heterogeneous mobile hardware.

General Conclusion

The work presented in this thesis addressed a central challenge of modern Big Data engineering: providing reliable, secure, and decentralized storage without depending on a costly centralized cloud infrastructure. Starting from the observation that smartphones collectively hold a considerable and largely unused storage capacity, we proposed **MobiCloud**, a distributed storage system that transforms a federation of ordinary smartphones into a collaborative, resilient, and privacy-preserving storage network.

Summary of the Work Carried Out

Throughout this thesis, we first established the conceptual foundations of distributed computing paradigms (Cloud, Edge, and Fog Computing) and of distributed storage systems. We then conducted a state-of-the-art review that revealed a clear research gap: no existing system simultaneously combines Big Data scalability, exclusive operation on mobile terminals, persistent storage through erasure coding, and modern resilience mechanisms.

Building on this analysis, we designed a complete architecture answering the five fundamental questions of any distributed storage system: who composes the network (membership and cluster formation), what is stored and how (adaptive Reed-Solomon erasure coding), where the data resides and how to find it (distributed catalog and gossip synchronization), how the system survives failures (reliability-aware election, heartbeat-based failure detection, and self-healing), and who may access the data (a zero-knowledge security pipeline). This design was formalized through a set of UML models covering the functional, structural, and behavioral views of the system.

Finally, we implemented the design as a real Android application, relying on a modern technology stack (Kotlin, Jetpack Compose, Hilt, Room, Ktor, the Android NDK for native erasure coding, and a React administration dashboard). The system was validated on a real cluster of seven smartphones.

Objectives Achieved

The tests carried out confirmed that the main objectives set at the beginning of this work were met:

- **Decentralization:** the system operates on ordinary smartphones with only a minimal relay server for signaling, which never stores user data.
- **Resilience:** file distribution and retrieval succeeded across the cluster, and the ($K = 4, N = 2$) erasure coding scheme tolerated the loss of nodes; cluster head failure was handled automatically by the Bully election mechanism while data remained fully available.

-
- **Security and privacy:** a zero-knowledge model was enforced through hardware-backed cryptographic identity (Android Keystore), AES-256-GCM fragment encryption, and ECDH-HKDF key distribution.
 - **Adaptability and transparency:** redundancy adapts to cluster reliability, and all distributed complexity is hidden behind a simple mobile interface.

Overall, MobiCloud demonstrates that the founding principles of Big Data storage, namely horizontal scalability across commodity nodes, erasure-coded redundancy, and decentralized coordination, can be applied effectively on heterogeneous and resource-constrained mobile hardware.

Perspectives

Although the proposed system fulfills its core objectives, several directions remain open for future work:

- **Extended performance evaluation:** the current validation involved seven devices. Further tests should assess the system at a larger scale, with a higher number of nodes, larger files, higher churn rates, and measurements of energy consumption, latency, and throughput under diverse network conditions, including large-scale simulation.
- **Embedded AI for predictive resilience:** a particularly promising direction is the integration of lightweight on-device machine learning models to predict node departures before they occur. By analyzing local metrics (battery level, connectivity history, usage patterns), the system could proactively redistribute fragments before a node disconnects, rather than reacting after the failure has happened.
- **User-configurable participation parameters:** the application could give the user finer control over their involvement in the network, for instance by choosing the conditions under which the device is allowed to become a cluster head (e.g. only while charging, on Wi-Fi, or above a given battery level), and defining when the application is active and contributes storage. This would better balance the coordination cost across users while respecting each device owner's preferences.
- **Post-partition consistency:** integrating CRDTs with tombstones would strengthen consistency guarantees when the network splits and later heals, preventing data resurrection after partition recovery.
- **Pure device-to-device communication:** extending the system to direct D2D protocols (such as Wi-Fi Direct) would further reduce its reliance on the relay server and enable operation in fully infrastructure-less environments.

In conclusion, this work constitutes a concrete step toward a viable, mobile-first alternative to centralized cloud storage. Beyond the prototype itself, it opens the way to intelligent, user-controlled, and fully decentralized storage networks built from the devices users already own.

Bibliography

- [1] Peter Mell and Timothy Grance. The nist definition of cloud computing, 2011-09-28 2011.
- [2] Cisco. What is edge computing, 2025. Consulté le 28 novembre 2025.
- [3] TechTarget. What is multi-access edge computing? benefits and use cases. <https://www.techtarget.com/searchnetworking/definition/multi-access-edge-computing-MEC>, 2024. Accessed: 2025.
- [4] Fortinet. What is edge computing? types, benefits & uses. <https://www.fortinet.com/resources/cyberglossary/what-is-edge-computing>, 2024. Accessed: 2025.
- [5] Michaela Iorga, Larry Feldman, Robert Barton, Michael J. Martin, Nedim S. Goren, and Charif Mahmoudi. Fog computing conceptual model. Technical Report NIST SP 500-325, National Institute of Standards and Technology, 2018.
- [6] Cloudian. Big data storage: Key components & top 6 storage technologies, n.d. Accessed: 2026-02-03.
- [7] SAP. What is big data?, n.d. Accessed: 2026-02-02.
- [8] Tom White. *Hadoop: The Definitive Guide*. O'Reilly Media, 4th edition, 2015.
- [9] Amazon s3. <https://aws.amazon.com/s3/>. Accessed: 2026-02-02.
- [10] Andrew S. Tanenbaum and Maarten Van Steen. *Distributed Systems: Principles and Paradigms*. Pearson, 2017.
- [11] George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair. *Distributed Systems: Concepts and Design*. Addison-Wesley, 5th edition, 2011.
- [12] H. S. Oluwatosin. Client-server model. *IOSR Journal of Computer Engineering*, 16:67–71, 2014.
- [13] Angela Bonifati, Panos K. Chrysanthis, Aris M. Ouksel, and Kai-Uwe Sattler. Distributed databases and peer-to-peer databases: Past and present. *ACM SIGMOD Record*, 37(1):5–11, 2008.
- [14] Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. Chord: A scalable peer-to-peer lookup service for internet applications. In *Proceedings of the ACM SIGCOMM Conference*, pages 149–160. ACM, 2001.

-
- [15] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the XOR metric. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, volume 2429 of *Lecture Notes in Computer Science*, pages 53–65. Springer, 2002.
- [16] Beverly Yang and Hector Garcia-Molina. Designing a super-peer network. In *Proceedings of the 19th International Conference on Data Engineering (ICDE '03)*, pages 49–60. IEEE, 2003.
- [17] IBM Think Insights. 10 industries that use distributed computing. <https://www.ibm.com/think/insights/distributed-computing-use-cases>, 2021. Accessed: 2026-02-02.
- [18] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.
- [19] Matei Zaharia et al. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.
- [20] Scalair. Object storage: Managing unstructured data at scale, 2018. Accessed: 2026-02-03.
- [21] Bastien Confais, Adrien Lèbre, and Benoît Parrein. Quelle place pour les systèmes de stockage objet dans une infrastructure de Fog/Edge computing ? In *Conférence d’informatique en Parallélisme, Architecture et Système (Compas)*, Lorient, France, 2016.
- [22] Bastien Confais, Adrien Lèbre, and Benoît Parrein. Performance analysis of object store systems in a fog and edge computing infrastructure. In *Transactions on Large-Scale Data- and Knowledge-Centered Systems XXXIII*, volume 10430 of *Lecture Notes in Computer Science*, pages 40–79. Springer, 2017.
- [23] Bastien Confais, Adrien Lèbre, and Benoît Parrein. An object store service for a Fog/Edge computing infrastructure based on IPFS and a scale-out NAS. In *Proceedings of the 1st IEEE International Conference on Fog and Edge Computing (ICFEC)*, pages 41–50. IEEE, 2017.
- [24] TechTerms. Dfs (distributed file system) definition, n.d. Accessed: 2026-02-03.
- [25] Techopedia. What is a distributed file system (dfs)?, n.d. Accessed: 2026-02-03.
- [26] Benjamin Depardon, Gaël Le Mahec, and Cyril Séguin. Analysis of six distributed file systems. Research Report hal-00789086, Université de Picardie Jules Verne / INRIA, 2013.
- [27] StoneFly. File storage vs. block storage vs. object storage, n.d. Accessed: 2026-02-03.

-
- [28] Qiang Zhu et al. Security and privacy in distributed systems. *Journal of Parallel and Distributed Computing*, 124:112–125, 2019.
- [29] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. The Google File System. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, pages 29–43. ACM, 2003.
- [30] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, pages 307–320. USENIX Association, 2006.
- [31] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, and Carlos Maltzahn. CRUSH: Controlled, scalable, decentralized placement of replicated data. In *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC '06)*. ACM, 2006.
- [32] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 205–220. ACM, 2007.
- [33] James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, and Andrew Warfield. Using lightweight formal methods to validate a key-value storage node in Amazon S3. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP '21)*, pages 836–850. ACM, 2021.
- [34] Andy Warfield. Building and operating a pretty big storage system (my adventures in Amazon S3). In *21st USENIX Conference on File and Storage Technologies (FAST '23)*, Santa Clara, CA, USA, 2023. USENIX Association.
- [35] Juan Benet. IPFS – content addressed, versioned, P2P file system. *arXiv preprint arXiv:1407.3561*, 2014.
- [36] Ricardo Monteiro, João Silva, João Lourenço, and Hervé Paulino. Decentralized storage for networks of hand-held devices. In *NOVA Laboratory for Computer Science and Informatics, Universidade Nova de Lisboa*, 2022. Preprint, Semantic Scholar.
- [37] Mehdi Assefi, Radu Stoleru, and Suman Bhunia. R-Drive: Resilient data storage and sharing for mobile edge clouds. *IEEE Transactions on Mobile Computing*, 2022.

-
- [38] Steven Huchton. A secure mobile distributed file system (MDFS). Master’s thesis, Naval Postgraduate School, Monterey, California, 2011. Defense Technical Information Center (DTIC).
- [39] Mohammad Sagor, Amran Haroon, Radu Stoleru, Suman Bhunia, Ala Altaweel, Mengyuan Chao, L. Jin, M. Maurice, and R. Blalock. DistressNet-NG: A resilient data storage and sharing framework for mobile edge computing in cyber-physical systems. *ACM Transactions on Cyber-Physical Systems*, 8(3), 2024.
- [40] Yacine Baza and Meriem Affaf Baza. Conception et développement d’un système de stockage dans le fog networking. Mémoire de master, spécialité ingénierie logicielle (il), Université des Sciences et de la Technologie Houari Boumediene (USTHB), Faculté d’Électronique et d’Informatique, Département Informatique, Alger, Algérie, 2018. Encadré par M. A. Djouama. Soutenu le 21/06/2018. Binôme N° 020/2018.
- [41] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. Research Report RR-7687, INRIA, 2011.
- [42] Paulo Sérgio Almeida, Ali Shoker, and Carlos Baquero. Delta state replicated data types. *Journal of Parallel and Distributed Computing*, 111:162–173, 2018.
- [43] Martin Kleppmann. Approaches to conflict-free replicated data types. *arXiv preprint*, 2024. arXiv:2409.12345.
- [44] A. Alahmadi et al. Proactive fault prevention using advanced machine learning strategies for cloud storage management. *ESR Groups Journal*, 2024.
- [45] Google. Meet android studio — android developers. <https://developer.android.com/studio/intro>. Accessed: 2026-06-02.
- [46] JetBrains. Kotlin programming language. <https://kotlinlang.org/>. Accessed: 2026-06-02.
- [47] Google. Jetpack compose — android’s modern toolkit for building native ui. <https://developer.android.com/develop/ui/compose>. Accessed: 2026-06-02.
- [48] Google. Dependency injection with hilt — android developers. <https://developer.android.com/training/dependency-injection/hilt-android>. Accessed: 2026-06-02.
- [49] Google. Save data in a local database using room — android developers. <https://developer.android.com/training/data-storage/room>. Accessed: 2026-06-02.
- [50] JetBrains. Ktor: Build asynchronous servers and clients in kotlin. <https://ktor.io/>. Accessed: 2026-06-02.

-
- [51] Google. Android ndk — android developers. <https://developer.android.com/ndk>.
Accessed: 2026-06-02.
- [52] Meta. React — the library for web and native user interfaces. <https://react.dev/>.
Accessed: 2026-06-02.

A Detailed Use Case Diagrams

This appendix presents the detailed use case diagrams of the six functional modules introduced in Section 4.6.

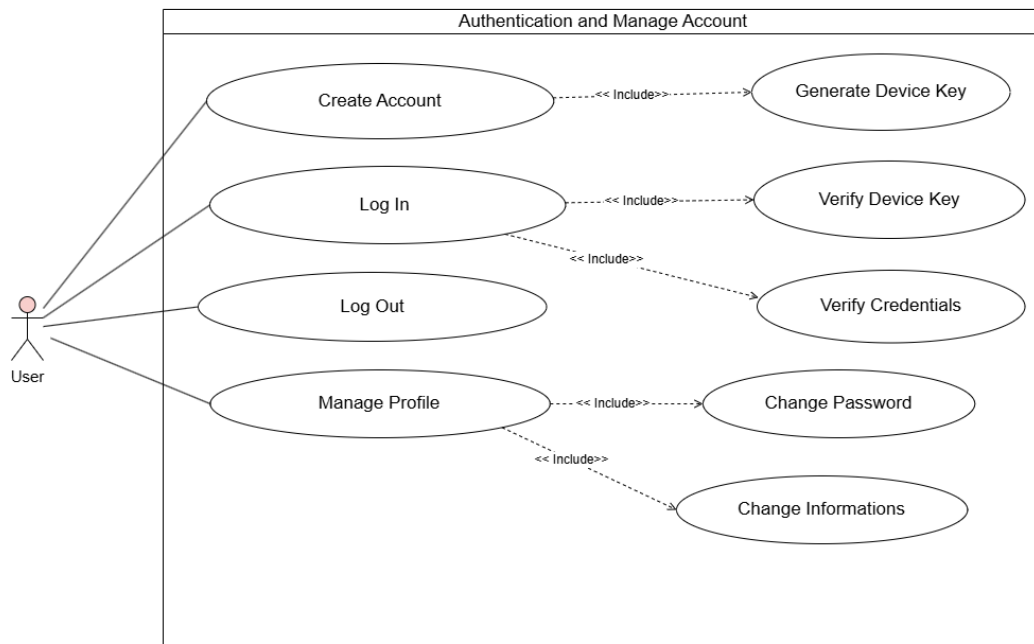


Figure A.1: Use Case Diagram of Authentication and Account Management.

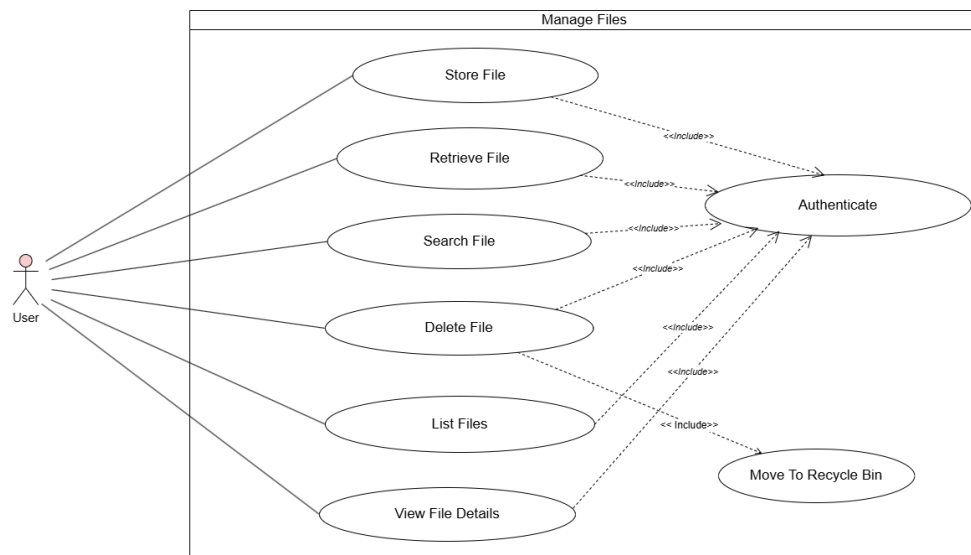


Figure A.2: Use Case Diagram of Manage Files.

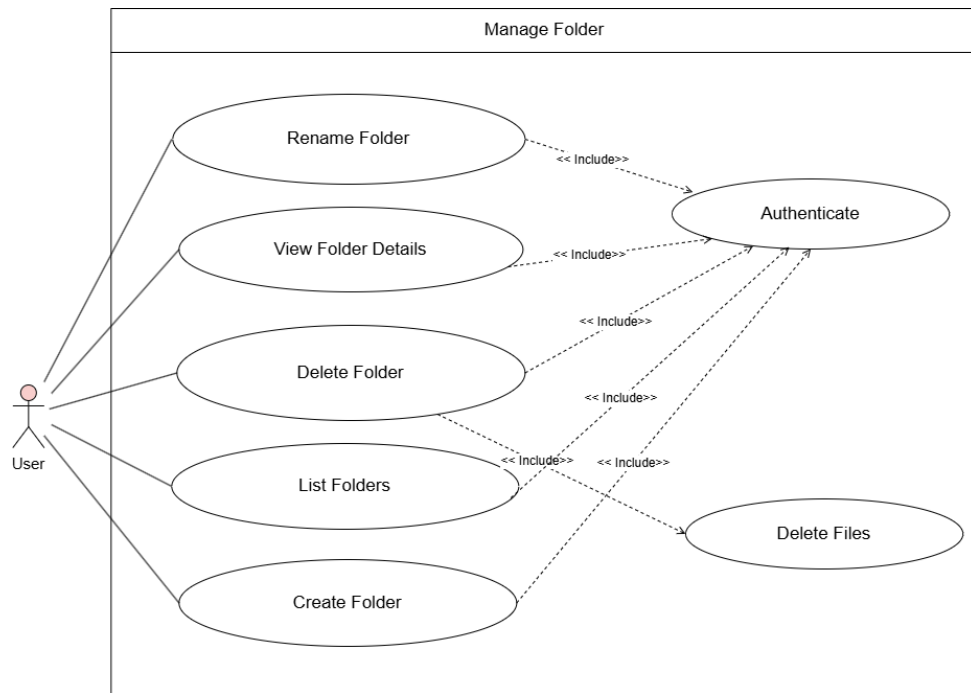


Figure A.3: Use Case Diagram of Manage Folders.

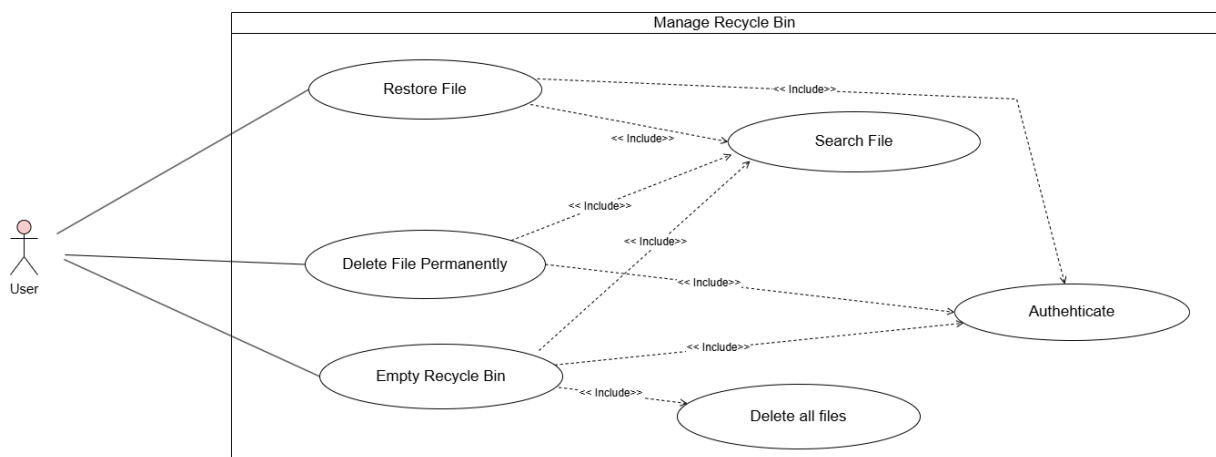


Figure A.4: Use Case Diagram of Recycle Bin.

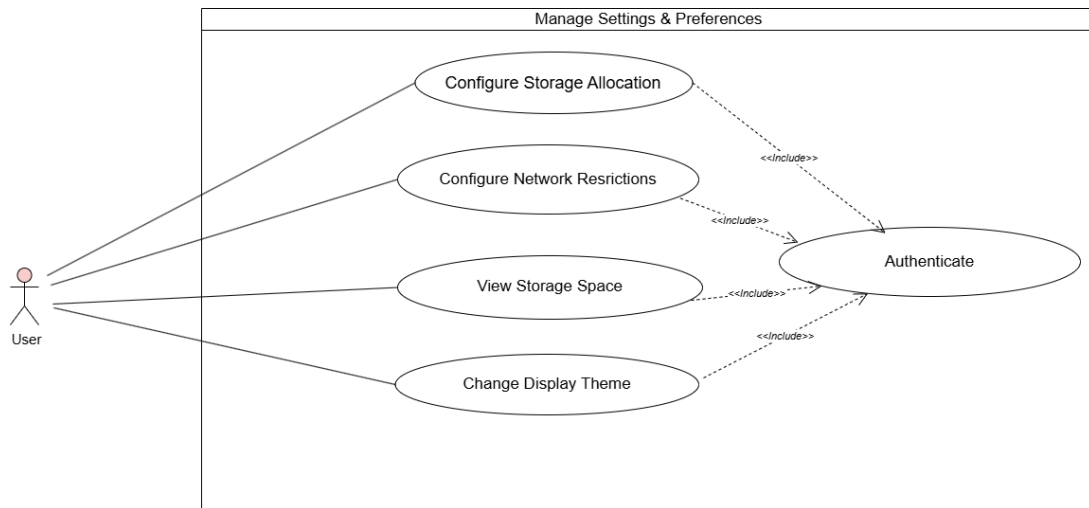


Figure A.5: Use Case Diagram of Manage Settings & Preferences.

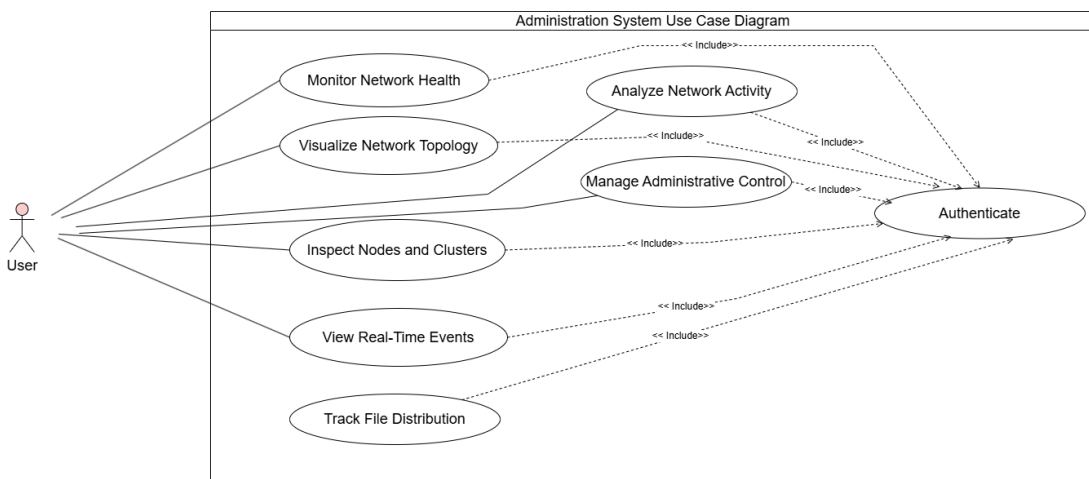


Figure A.6: Use Case Diagram of Administrator System.

B Detailed Scenario Descriptions

File Distribution
Identification Summary (Normal Flow)
Title: File Distribution Goal: allows the user to store and distribute a file across the network. Actor(s): user.
Pre-conditions
• The user must be authenticated and possess a cryptographic identity. • The local node must be attached to a cluster. • The peer discovery and distribution processes must be running.
Flow Description
• The user accesses the application and selects the file to store; • The node retrieves the list of available peers from the local database; If <code>reachablePeers</code> $> k$ Then * Filter peers with sufficient free storage: <code>freeStorage</code> $\geq \lceil \text{fileSize}/k \rceil + 100$ MB; If <code>capablePeers</code> $\geq k$ Then - Compute the parity count n based on the weakest reliability score; - Adapt n and select the top $k + n$ host peers; - Split the file into k data blocks and n parity blocks (erasure coding); - Encrypt each fragment and protect the master key for the owner; - Distribute the encrypted fragments in parallel to the selected peers; If <code>dataConfirmed</code> $\geq k$ And <code>parityConfirmed</code> $\geq \lceil n/2 \rceil$ Then - Save the file map (catalog entry) in the local database; - Index each confirmed fragment in the DHT; - Propagate the file map to neighboring peers via gossip; - Notify the user that the file has been successfully stored. Else Execute [Exception 3]. Else Execute [Exception 2]. Else Execute [Exception 1].
Exceptions (Alternative Flow)
• Exception 1: the system displays an error message indicating that the network does not have enough available peers to guarantee redundancy. • Exception 2: the system displays an error message indicating that the global storage capacity is insufficient to host the file. • Exception 3: the system displays an error message indicating that the storage quorum was not reached; no entry is saved in the catalog or DHT, and gossip propagation is not triggered.

Tableau B.1: Textual description of the File Distribution scenario.

File Retrieval
Identification Summary (Normal Flow)
Title: File Retrieval Goal: allows the user to retrieve, reconstruct and decrypt a file previously stored on the network. Actor(s): user (file owner).
Pre-conditions
• The user must be authenticated and possess the private decryption key. • The file must be referenced in the local catalog. • At least k fragments must still be accessible on the network.
Flow Description
• The user accesses the application and selects the file to retrieve; • The system queries the local catalog to obtain the file information (fragments list, parameters k and n, protected master key); • The system locates the active host peers for each fragment of the file; If localizedFragments $\geq k$ Then * Build a download pool of $\min(\text{localizedFragments}, k + 2)$ locations sorted by reliability; * Launch the parallel download of the fragments from the selected peers; If receivedFragments $\geq k$ Then - Cancel the remaining download jobs; - Unwrap the master key using the owner's private key; - Decrypt each downloaded fragment with its derived key; - If parity fragments were used, decode the file via erasure coding; - Reconstruct the file, verify its integrity (cryptographic hash); If reconstruction successful Then - Atomically write the file to the destination directory; - Notify the user that the file is available, providing its path. Else Execute [Exception 3]. Else Execute [Exception 2]. Else Execute [Exception 1].
Exceptions (Alternative Flow)
• Exception 1: the system displays an error message indicating that fewer than k fragments could be located in the network; the reconstruction is impossible because too many host peers are unreachable. • Exception 2: the system displays an error message indicating that the download is incomplete; fewer than k fragments could be retrieved successfully and no reconstruction is attempted. • Exception 3: the system displays an error message indicating that the file is corrupted or that the decryption key is invalid; the partial file is discarded and never written to the destination directory.

Tableau B.2: Textual description of the File Retrieval scenario.

C Relay Server Administration Dashboard

The React dashboard is a full-screen real-time administration interface for the relay server deployed on Render (Frankfurt). It is structured around three zones: a fixed header bar, a main two-column panel, and a bottom strip.

Header bar. The header displays the relay connectivity status (green dot when reachable, red otherwise), a live session and node count, the 5-minute churn rate with a warning when it exceeds 30%, a *Reset nodes* admin action that disconnects all sessions (protected by an ADMIN_SECRET prompt), and a dark/light theme toggle.

Interactive topology graph. The central panel renders an interactive 2D network graph using Cytoscape.js with the cose-bilkent layout. Each cluster is drawn as a coloured compound boundary; Cluster-head appear as larger gold nodes, regular members as smaller coloured nodes, and recently departed nodes as grey dashed outlines. Intra-cluster edges form a full mesh between connected members. When a data block, election message, or gossip signal is relayed, an animated particle travels along the corresponding edge in real time (gold for Bully election, cyan for gossip signals, cluster colour for data blocks).

Cluster tooltip. Clicking a cluster compound node opens a contextual tooltip (Figure C.1) reporting the cluster identifier, the number of connected members out of total, aggregate storage usage with a fill bar, and the average reliability score of all members.

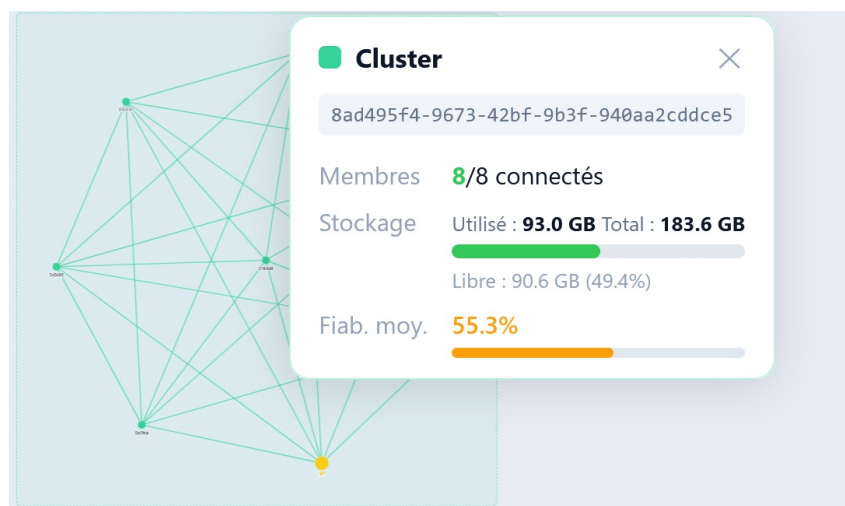


Figure C.1: Cluster tooltip.

Right panel. The right column stacks four sub-panels vertically: four KPI sections (*Cluster*, *Network*, *Security*, *Stability*) sourced from the relay API; a collapsible Cluster panel listing

active clusters with Cluster-head identity, free storage, and average reliability; a Fragment Map showing per-file availability status; and a fragment navigation bar for stepping through fragment locations on the graph. Figure C.2 shows the complete dashboard in its default state.

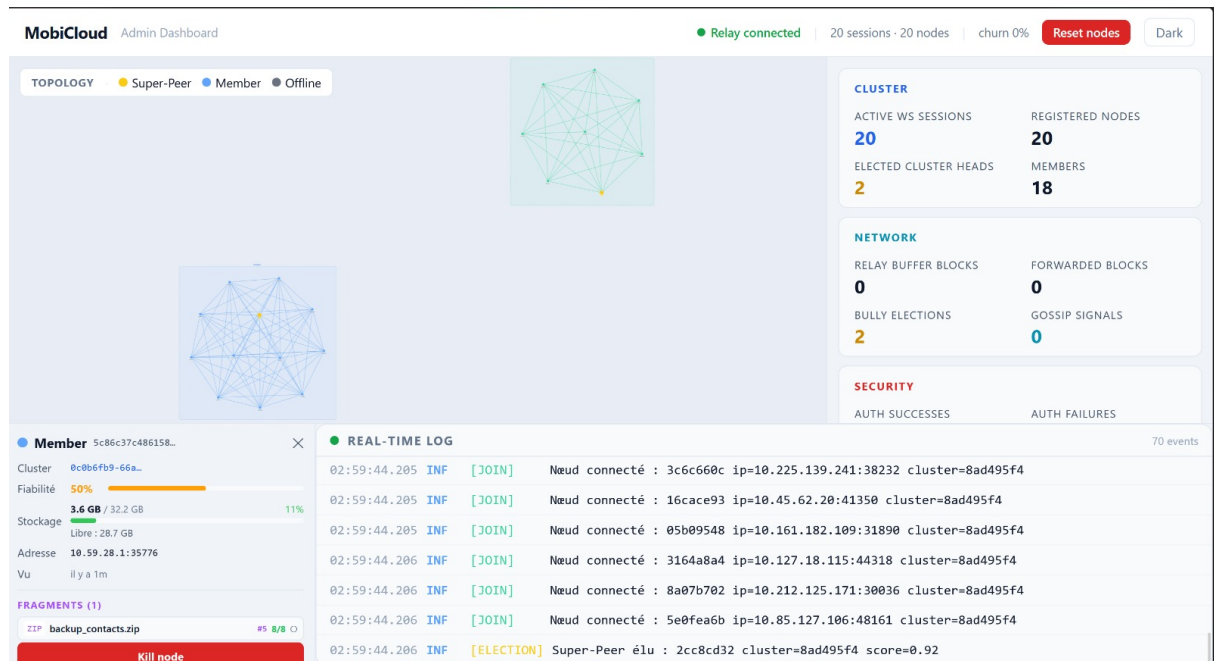


Figure C.2: Dashboard.

Node detail panel (bottom-left strip). Clicking a node in the graph replaces the bottom-left network chart with an inspection panel (Figure C.3). It reports the node role (Cluster-head or Member), its truncated nodeId, reliability score with a colour-coded progress bar, storage usage (used/total/free) with a fill bar, IP address, port, and time since last heartbeat. Any fragments hosted on this node are listed with their file name, fragment index, and replication health. A *Kill node* admin button (secret cached in sessionStorage) forces an immediate disconnection, simulating a node failure.

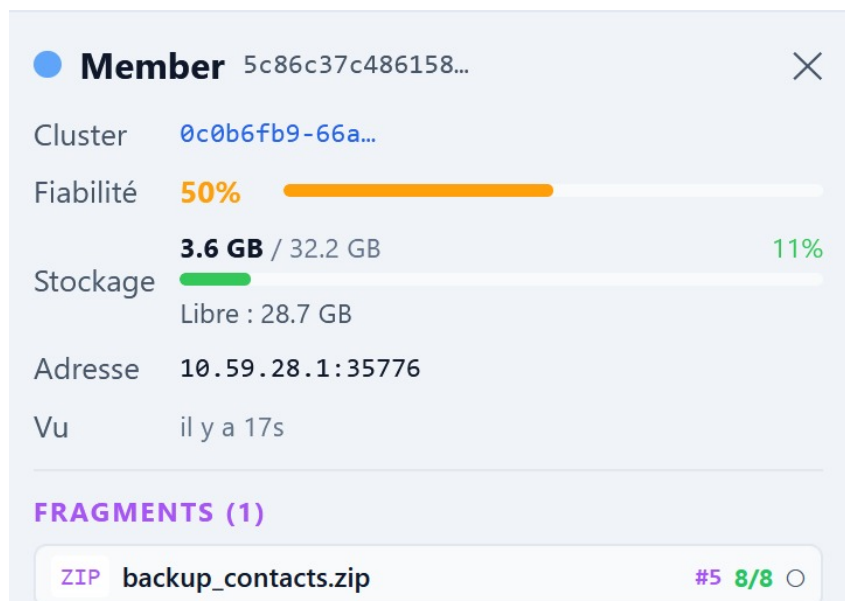


Figure C.3: Node detail panel.

Fragment Map. The Fragment Map panel (Figure C.4), sourced from `/metrics/fragments` and polled every 5 seconds, lists every distributed file with its name, MIME type, size, and a colour-coded availability status: *SECURED* (green) if at least $k=4$ fragments are online, *DEGRADED* (orange) if online count is between k and n , and *LOST* (red) if fewer than k fragments are reachable. Expanding a file row reveals all n fragment tiles, each showing its index, the hosting node's truncated nodeId, and its connectivity state.

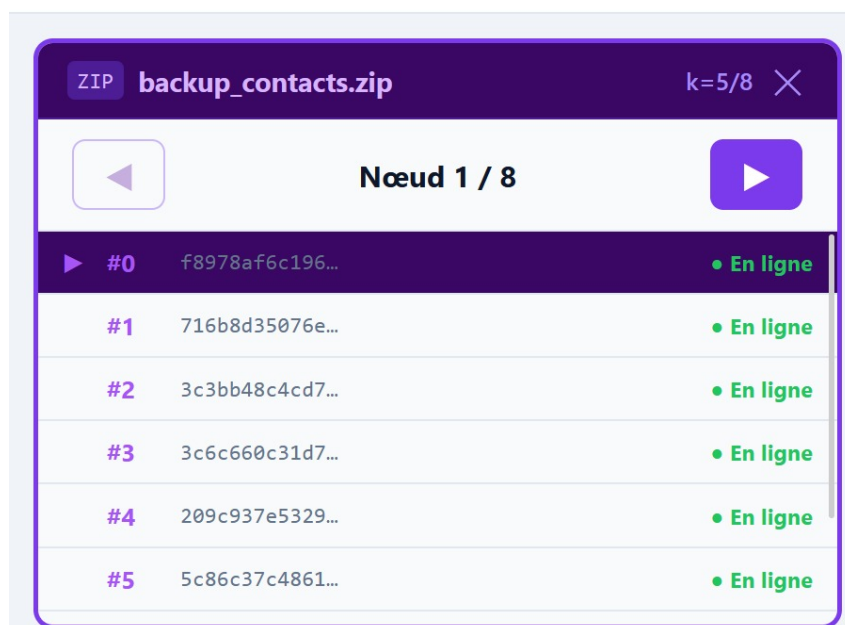


Figure C.4: Fragment Map.

Real-time log panel. The full-width bottom-right area (Figure C.5) displays a live scrolling log.

● REAL-TIME LOG				300 events
03:10:14.244	INF	[JOIN]	Nœud connecté : 16cace93 ip=10.45.62.20:41350 cluster=8ad495f4	
03:10:14.244	INF	[JOIN]	Nœud connecté : 05b09548 ip=10.161.182.109:31890 cluster=8ad495f4	
03:10:14.244	INF	[JOIN]	Nœud connecté : 3164a8a4 ip=10.127.18.115:44318 cluster=8ad495f4	
03:10:14.244	INF	[JOIN]	Nœud connecté : 8a07b702 ip=10.212.125.171:30036 cluster=8ad495f4	
03:10:14.244	INF	[JOIN]	Nœud connecté : 5e0fea6b ip=10.85.127.106:48161 cluster=8ad495f4	
03:10:14.244	INF	[ELECTION]	Super-Peer élu : 2cc8cd32 cluster=8ad495f4 score=0.92	
03:11:44.250	INF	[JOIN]	Nœud connecté : 4babd634 ip=10.193.192.136:46292 cluster=0c0b6fb9	

Figure C.5: Real-time log panel.